

**PENERAPAN DYNAMIC DIALOGUE BERBASIS
MULTI-AGENT PADA NPC DI GAME
“TEA’N BREW”**

SKRIPSI

Diajukan sebagai salah satu syarat untuk memperoleh gelar Sarjana (S1)

**STEVANUS RYO WIJAYA
10122014**



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS KOMPUTER INDONESIA
2026**

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Maksud dan Tujuan	3
1.4 Batasan Masalah	3
1.5 Metodologi Penelitian	3
1.6 Sistematika Penulisan	4
BAB 2 TINJAUAN PUSTAKA	6
2.1 <i>Game</i>	6
2.1.1 Endless Game	6
2.2 <i>Visual Novel</i>	6
2.3 NPC	7
2.4 <i>Game Experience Questionnaire (GEQ)</i>	7
2.5 <i>Narrative Engagement Scale (NES)</i>	9
2.6 <i>Dynamic Dialogue</i>	10
2.7 <i>Large Language Model (LLM)</i>	10
2.8 <i>Prompt Engineering</i>	10
2.8.2 <i>System Prompt</i>	11
2.8.3 <i>User Prompt</i>	12
2.9 <i>AI Agent</i>	12
2.10 <i>Multi Agent System</i>	14
2.10.1 Autonomous	14
2.10.2 Pre Define	15
2.10.3 LangGraph	15
2.10.4 Actor-Critic Agent	16
2.11 OCEAN	16
2.12 Unity Engine	21
2.13 Python	21

BAB 3 ANALISIS DAN PERANCANGAN	23
3.1 Analisis Sistem.....	23
3.1.1 Domain Understanding	23
3.1.2 Persiapan Data.....	32
3.1.3 Sistem Multi Agent	34
3.1.4 Analisis Kebutuhan Fungsional	53
3.1.5 Analisis Kebutuhan Non Fungsional	58
3.2 Perancangan Sistem	63
DAFTAR PUSTAKA	65
LAMPIRAN A Review dan Rating Pemain	68
LAMPIRAN B Kuisisioner GEQ dan NES.....	69

DAFTAR GAMBAR

Gambar 3.1 <i>User flow</i> skenario menu.....	24
Gambae 3.2 <i>User flow</i> skenario customer came	25
Gambar 3.3 Mekanik <i>randomize customer</i>	26
Gambar 3.4 <i>User flow</i> skenario make tea	27
Gambar 3.5 <i>User flow</i> skenario curhat session	28
Gambar 3.6 <i>User flow</i> skenario daily cycle	28
Gambar 3.7 <i>User flow</i> skenario game over	29
Gambar 3.8 Alur sistem multi agent	53
Gambar 3.9 <i>Use Case Diagram</i>	54
Gambar 3.10 Blok Diagram Arsitektur Sistem.....	Error! Bookmark not defined.

DAFTAR TABEL

Tabel 3.1 List skenario game “ <i>Tea ’n Brew</i> ”	23
Tabel 3.2 Daftar Resep Minuman	24
Tabel 3.3 List <i>Profile Customer</i> di <i>Game</i>	26
Tabel 3.5 Hasil kuisisioner GEQ	30
Tabel 3.6 Hasil kuisisioner NES	30
Tabel 3.7 Pemetaan mekanik <i>randomize customer</i> terhadap masalah	31
Tabel 3.8 Data yang dibutuhkan	32
Tabel 3.9 Pemetaan masalah ke <i>agent</i>	34
Tabel 3.10 Peforma model untuk <i>profile agent</i>	35
Tabel 3.11 Pemetaan <i>system prompt profile agent</i>	35
Tabel 3.12 Pemetaan <i>user prompt profile agent</i>	37
Tabel 3.13 Jenis dialog pada game “ <i>Tea ’n Brew</i> ”	39
Tabel 3.14 Peforma model untuk <i>dialogue agent</i>	39
Tabel 3.15 Pemetaan <i>system prompt dialogue agent</i>	40
Tabel 3.16 Pemetaan <i>user prompt order dialogue</i>	41
Tabel 3.17 Pemetaan <i>user prompt False Order Dialogue</i>	42
Tabel 3.18 Pemetaan <i>user prompt Failed Order Dialogue</i>	43
Tabel 3.19 Pemetaan <i>user prompt Success Order Dialogue</i>	44
Tabel 3.20 Pemetaan <i>user prompt Venting Dialogue</i>	45
Tabel 3.21 Pemetaan <i>user prompt Venting Reaction Dialogue</i>	47
Tabel 3.22 Pemetaan <i>user prompt Venting Closing Dialogue</i>	48
Tabel 3.23 Peforma model untuk <i>critic agent</i>	50
Tabel 3.24 Pemetaan <i>user prompt Venting Reaction Dialogue</i>	50
Tabel 3.25 Pemetaan <i>user prompt Venting Reaction Dialogue</i>	51
Tabel 3.26 Definisi Aktor	54
Tabel 3.27 Definisi Use Case	54
Tabel 3.28 Use Case Scenario Melayani <i>Customer</i>	55
Tabel 3.29 Use Case Scenario Membuat <i>Profile Customer</i>	56
Tabel 3.30 Use Case Scenario Membuat Dialog <i>Customer</i>	56

Tabel 3.31 Use Case Scenario Membuat Dialog <i>Customer</i>	57
Tabel 3.32 Struktur Atribut Class <i>ProfileResponseError!</i> Bookmark not defined.	
Tabel 3.33 Struktur Atribut Class <i>DialogResponseError!</i> Bookmark not defined.	
Tabel 3.34 Struktur Atribut Class <i>EvaluateResponseError!</i> Bookmark not defined.	

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Video Game telah berkembang dari yang awalnya berupa permainan sederhana menjadi sebuah produk hiburan yang menonjolkan kompleksitas mekanik dan kedalaman cerita. Hal tersebut membuat elemen narasi menjadi salah satu fondasi utama keberhasilan judul-judul *story-driven game* modern [1]. Penelitian menunjukkan bahwa kualitas narasi dan dialog berkontribusi langsung pada peningkatan *immersion* dan keterikatan emosional pemain terhadap dunia permainan, sehingga *game* yang memiliki narasi baik cenderung menghasilkan pengalaman bermain yang jauh lebih mendalam [1], [2]. Untuk menyampaikan narasi secara efektif, industri *game* umumnya memanfaatkan *dialogue system* sebagai sarana penyampaian utama karena dialog dapat mencakup motivasi karakter, penjelasan plot cerita, dan pemberian opsi atau cabang narasi kepada pemain [3], [4]. Sebagian besar industri *game* saat ini masih mengimplementasikan *dialogue system* berbasis *script* atau *dialog-tree*, hal ini dapat mengakibatkan dialog yang disampaikan seringkali mengurangi *immersion* dan kedalaman emosi pemainnya. Kondisi ini dapat mengurangi *sense of presence* pada pemain dan keterikatan emosional pada *game* tersebut [5], [6].

Penelitian ini dilakukan pada *game* berjudul “*Tea’n Brew*” yang merupakan *game visual novel* dimana pemain akan berperan sebagai penjaga *cafe* yang harus melayani serta mendengarkan dan memberi jawaban dari curhatan setiap *customernya* yang akan berpengaruh pada *mood customer* tersebut. Sistem *game* saat ini masih menggunakan *dialogue system* tradisional yaitu dengan *script* yang ditulis manual oleh *developer game* tersebut. Pendekatan sistem tersebut membuat dialog *game* menjadi statis dan mudah ditebak sehingga mengurangi tingkat *immersion* dan kedalaman emosi cerita terhadap pemain. Hal ini dapat dibuktikan dari rendahnya *rating game* yaitu hanya di 3.06 dan *review* para pemain yang berfokus pada dialog yang statis dan kurangnya variasi.

Evaluasi pengalaman pemain menggunakan *Game Experience Questionnaire* (GEQ) yang dimodifikasi dengan *Narrative Engagement Scale* (NES) yang terdiri dari 44 pertanyaan dengan skala likert 1-7 terhadap 30 responden difokuskan pada dimensi yang berkaitan langsung dengan kualitas penyampaian narasi [7], [8]. Hasil menunjukkan *score Narrative Presence* (57.86%) sedang, *Attentional Focus* (46.10%) sedang, dan *Emotional Engagement* (49.81%) sedang, sedangkan untuk dimensi *Narrative Understanding* (69.43%) tinggi. Pola ini mengindikasikan penyampaian dialog saat ini kurang berhasil membawa pemain ke dalam cerita *game*, hal tersebut dapat terlihat dari dimensi *Immersion* (52.38%) yang masuk ke kategori sedang, tidak tinggi. Temuan ini menegaskan bahwa perlunya ada perbaikan pada fitur dialog saat ini untuk meningkatkan kualitas narasi pada *game* agar dapat lebih adaptif dan personal pada pemain sehingga dapat meningkatkan *immersion*, *emotional engagement*, *attentional focus* dan *narrative presence* pada pemain.

Berdasarkan permasalahan tersebut, diperlukan adanya *dynamic dialogue* yang personal pada pemain dan dapat menggantikan sistem *script* tradisional yang terbukti kurang efektif untuk menyampaikan dialog. Oleh karena itu, sistem *Multi-Agent* dipilih sebagai solusi karena sistem dialog pada *game* "Tea'n Brew" membutuhkan dua tugas yang berbeda namun saling bergantung secara sekuensial yaitu membuat *profile customer*, dan menghasilkan dialog yang konsisten dengan *profile* tersebut sambil mengakses konteks permainan melalui *tools* di Unity Engine. Membebankan kedua tanggung jawab ini pada satu LLM berisiko menghasilkan output yang tidak konsisten karena model harus mengelola konteks yang kompleks secara bersamaan. Arsitektur *Multi Agent* memungkinkan dekomposisi tugas ini ke dalam *agent* dengan peran yang lebih spesifik [9]. Selain itu, dengan memisahkan *actor agent* yang bertugas menghasilkan dialog dan *critic agent* yang bertugas mengevaluasi konsistensi dan kualitasnya, sistem dapat memastikan output yang dihasilkan lebih personal dan relevan terhadap *profile customer*.

1.2 Rumusan Masalah

Berdasarkan penjelasan pada latar belakang di atas, penelitian ini merumuskan masalah yaitu apakah penerapan sistem *dynamic dialogue* berbasis *multi agent* dapat meningkatkan *immersion*, *emotional engagement*, *attentional focus* dan *narrative presence* pemain terhadap game “Tea 'n Brew”?

1.3 Maksud dan Tujuan

Maksud penelitian ini adalah untuk menerapkan sistem *dynamic dialogue* berbasis *multi agent* yang dapat menghasilkan dialog yang lebih terpersonalisasi agar tingkat *immersion*, *emotional engagement*, *attentional focus* dan *narrative presence* meningkat. Tujuan penelitian ini untuk membantu meningkatkan tingkat *immersion*, *attentional focus*, *emotional engagement*, dan *narrative presence* agar pemain lebih merasakan penyampaian narasi dalam dialog lebih personal sehingga keterikatan emosional pemain terhadap dunia *game* meningkat.

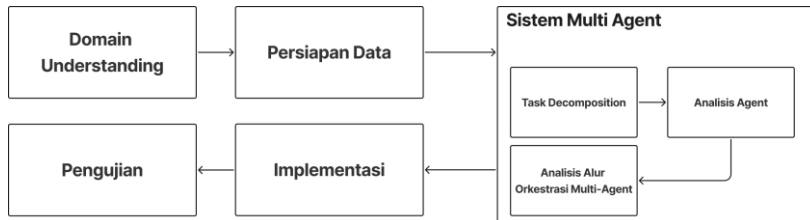
1.4 Batasan Masalah

Untuk menjaga fokus dan keterjangkauan penelitian, ruang lingkup dibatasi sebagai berikut:

1. Fokus penelitian ini hanya pada bagian dialog dan *customer* acak di game “Tea 'n Brew”
2. Validasi pengalaman pemain menggunakan *Game Experience Questionnaire* (GEQ) dan *Narrative Engagement Scale* (NES) untuk validasi lebih mendalam tentang narasi
3. Penelitian dilakukan menggunakan Unity Engine sebagai platform pengembangan *game* dan C# sebagai bahasa pemrograman.
4. Pembuatan sistem *multi agent* akan menggunakan bahasa pemrograman python.

1.5 Metodologi Penelitian

Metodologi penelitian yang digunakan mengacu pada tahapan perancangan *multi agent* yang disesuaikan dengan penelitian yang dilakukan. Urutan dari penelitian digambarkan pada **Gambar 1.1**.



Gambar 1.1 Metodologi Penelitian

1. Domain Understanding

Pada tahapan ini dilakukan pengumpulan informasi melalui validasi masalah, dan pengujian menggunakan kuisioner GEQ dan NES untuk memahami permasalahan pada domain kasus penelitian ini. Pemahaman tentang domain ini menjadi dasar untuk merancang sistem *multi agent* yang cocok.

2. Persiapan Data

Pada tahap ini dilakukan persiapan data apa saja yang akan dipakai dalam membangun sistem *multi agent*.

3. Multi Agent

Pada tahapan ini dilakukan untuk merancang sistem *multi agent* untuk menghasilkan dialog yang dinamis yang konsisten antara *profile customer* dan dialognya.

4. Implementasi

Tahapan ini dilakukan dengan mengembangkan sistem dialog pada *game visual novel* berjudul “*Tea 'n Brew*”.

5. Pengujian

Pada tahap ini dilakukan pengujian *game visual novel* “*Tea 'n Brew*” yang telah melalui seluruh tahapan pada metodologi ini dengan melakukan *post-test* pada para pemain.

1.6 Sistematika Penulisan

Sistematika penulisan dalam penelitian ini dibagi menjadi 5 bab yang tersusun secara sistematis untuk memberikan gambaran umum mengenai penelitian yang dikerjakan.

BAB 1 Pendahuluan

Pada bab ini akan menjelaskan mengenai latar belakang, perumusan masalah, maksud dan tujuan, batasan masalah, metodologi penelitian dan juga sistematika penulisan.

BAB 2 Landasan Teori

Pada bab ini akan membahas tentang uraian hasil yang di dapatkan pada penelitian yang dilakukan, konsep dasar, penelitian terdahulu, dan teori dari para ahli yang berkaitan dengan penelitian. Meninjau permasalahan dan hal-hal yang berguna dari penelitian-penelitian serupa yang pernah dikerjakan sebelumnya, lalu menggunakannya untuk acuan dalam pemecahan masalah pada penelitian ini.

BAB 3 Analisis dan Perancangan Sistem

Pada bab ini akan membahas tentang analisis sistem yang sudah berjalan, menjabarkan masalah yang ada, lalu merancang solusi untuk mengatasi permasalahan tersebut.

BAB 4 Implementasi dan Pengujian Sistem

Bab ini akan membahas tentang proses implementasi solusi terhadap sistem yang sudah berjalan sebelumnya.

BAB 5 Kesimpulan dan Saran

Pada bab terakhir ini akan membahas tentang kesimpulan dari hasil penelitian dan juga memberikan saran untuk pengembangan selanjutnya

BAB 2

TINJAUAN PUSTAKA

2.1 Game

Video game merupakan produk hiburan modern yang mengintegrasikan kompleksitas mekanik dengan kedalaman narasi sebagai fondasi utamanya. Di dalam ekosistem permainan yang berbasis cerita (*story-driven games*), elemen naratif berfungsi sebagai instrumen krusial untuk membangun dunia yang koheren dan menyampaikan motivasi karakter kepada pemain. Keberhasilan sebuah judul dalam medium ini sering kali ditentukan oleh sejauh mana desain narasi dan *gameplay* dapat berjalan beriringan untuk menciptakan pengalaman yang mendalam [1].

2.1.1 Endless Game

Endless gameplay merupakan sebuah model desain permainan di mana sistem dirancang tanpa memiliki kondisi kemenangan akhir (*winning condition*) yang absolut, sehingga permainan dapat berjalan terus-menerus selama pemain mampu bertahan. Karakteristik utama dari *endless game* terletak pada interaksi kontrol yang berkesinambungan serta penerapan sistem tingkat kesulitan yang meningkat secara progresif (*progressive difficulty adjustment*) seiring bertambahnya waktu atau jarak yang ditempuh pemain. Fokus utama dari mekanik ini adalah menguji daya tahan (*endurance*) dan ketangkasan pemain dalam merespons tantangan yang semakin cepat atau kompleks. Melalui struktur permainan yang tidak terbatas ini, *endless game* mampu menciptakan siklus permainan yang adiktif serta memotivasi pemain untuk terus memperbaiki pencapaian *score* tertinggi (*high score*) mereka pada sesi-sesi permainan berikutnya [10].

2.2 Visual Novel

Visual Novel merupakan sebuah genre dari fiksi interaktif yang menggabungkan narasi berbentuk teks dengan elemen visual seperti gambar. Fokus utama dari genre ini terletak pada penyampaian cerita melalui dialog yang di mana interaksi pemain biasanya terbatas pada pengambilan keputusan yang

menentukan arah cabang narasi. Oleh karena itu, nilai utama (*core value*) dari sebuah *Visual Novel* terletak pada aspek *immersion*-nya, di mana kekuatan narasi dan estetika visual harus mampu membuat pemain terhanyut secara emosional dan kognitif ke dalam dunia fiksi tersebut. Dalam perkembangannya, *Visual Novel* sering digunakan sebagai studi kasus untuk memahami bagaimana keterikatan naratif dan elemen mekanik dalam *game* berbasis peran dapat memengaruhi pengalaman pemain secara keseluruhan [11].

2.3 NPC

Non-Playable Character (NPC) merupakan karakter dalam video game yang tidak dikendalikan secara langsung oleh pemain, melainkan digerakkan secara otomatis oleh sistem komputer melalui algoritma kecerdasan buatan (*Artificial Intelligence*). Dalam struktur narasi dan mekanik permainan, NPC memegang peranan krusial untuk membangun lingkungan yang dinamis, berfungsi sebagai sekutu, musuh, pemberi misi (*quest giver*), ataupun elemen figuran untuk menghidupkan dunia *game*. Kualitas desain sebuah NPC sangat bergantung pada aspek keterpercayaan (*believability*), di mana perilaku, pola interaksi, dan respons visual karakter tersebut harus mampu menyelaraskan diri dengan ekspektasi pemain guna mempertahankan imersi (*immersion*) di dalam permainan. Ketika diintegrasikan ke dalam game berbasis agen cerdas (*intelligent agents*), NPC tidak lagi bergerak menggunakan skrip kaku, melainkan mampu mengevaluasi kondisi sekitar dan mengambil keputusan secara mandiri untuk berinteraksi dengan karakter lain maupun pemain secara adaptif [12].

2.4 Game Experience Questionnaire (GEQ)

Game Experience Questionnaire (GEQ) merupakan metode evaluasi yang digunakan untuk mengukur berbagai dimensi pengalaman pemain saat berinteraksi dengan sebuah sistem permainan. Metrik ini sangat efektif untuk memvalidasi model pengalaman dalam permainan (seperti *immersion* dan *flow*) guna memastikan bahwa mekanisme yang diimplementasikan telah mencapai target kepuasan pengguna yang diinginkan. Dalam konteks penelitian *game*, GEQ sering diterapkan sebagai standar untuk melakukan validasi terhadap pengalaman pemain secara menyeluruh [7].

Berikut ini adalah penjelasan masing-masing dimensi dari GEQ:

1. *Immersion* (Imersi) Komponen *immersion* merupakan bagian dari Modul Inti (*Core Module*) pada GEQ yang berfungsi untuk mengukur seberapa dalam perasaan pemain ketika sedang memainkan *game* tersebut. Dimensi ini mengevaluasi seberapa jauh pemain merasa terhanyut ke dalam lingkungan permainan.
2. *Flow* (Kondisi Fokus Penuh) Berdasarkan paper tersebut, *flow* didefinisikan (mengutip konsep dari Csikszentmihalyi) sebagai sebuah pengalaman optimal yang memotivasi secara intrinsik, yang diinduksi oleh sebuah aktivitas di mana seseorang merasa terserap atau fokus sepenuhnya (*fully absorbed*). Dalam konteks *game*, ini adalah kondisi ketika pemain sangat terfokus dan menikmati alur permainan.
3. *Competence* (Kompetensi) Komponen *competence* digunakan untuk mengevaluasi perasaan mampu, ahli, dan kompeten dari diri pemain saat mereka memainkan *game*. Paper ini secara khusus menemukan bahwa *competence* memiliki hubungan teoritis yang sangat kuat dengan *usability* (kebolegunaan/kemudahan navigasi) dari *game* tersebut. *Usability* yang baik dianggap sebagai prasyarat dasar agar pemain dapat merasa kompeten dan mendapatkan pengalaman bermain yang baik.
4. *Positive Affect* (Emosi Positif) Dimensi *positive affect* juga termasuk di dalam Modul Inti dan digunakan untuk mengukur perasaan positif atau kesenangan yang muncul saat pemain berinteraksi dengan *game*. Komponen ini mewakili emosi-emosi baik yang dirasakan selama permainan berlangsung.
5. *Tension / Annoyance* (Ketegangan / Rasa Jengkel) Meskipun kata *Tension* tidak dipakai di paper ini, paper ini mengukur dimensi sejenis melalui komponen *Annoyance* (rasa jengkel) dan *Negative Affect* (emosi negatif). Komponen ini berada di dalam Modul Inti GEQ yang bertujuan untuk mengukur perasaan negatif, rasa frustrasi, atau gangguan yang dialami pemain saat bermain *game*. Jika Anda diwajibkan menggunakan kata *Tension*, Anda dapat menjelaskan bahwa komponen ini mengukur emosi negatif dan hambatan psikologis pemain

seperti yang direpresentasikan oleh dimensi *Negative Affect* dan *Annoyance* di dalam penelitian tersebut.

2.5 Narrative Engagement Scale (NES)

Narrative Engagement Scale (NES) merupakan instrumen pengukuran psikologis yang dirancang untuk mengevaluasi tingkat keterlibatan mental dan emosional individu terhadap sebuah narasi. Metrik ini digunakan untuk memahami sejauh mana pemain merasa terikat dengan alur cerita, karakter, serta dunia permainan yang disajikan. Dalam konteks *video game*, NES menjadi alat ukur yang krusial untuk memvalidasi apakah narasi berhasil memberikan pengalaman narasi pada pemain seperti keterikatan emosional [8].

Berikut ini adalah penjelasan masing-masing dimensi dari NES:

1. *Narrative Understanding* (Pemahaman Naratif) Dimensi ini berkaitan dengan seberapa mudah audiens memahami dan memaknai sebuah narasi. Dari perspektif model mental, dimensi ini mengukur kemudahan audiens dalam mengonstruksi model makna dari cerita tersebut. Pemahaman naratif bekerja secara asimetris: audiens yang benar-benar terlibat umumnya tidak menyadari bahwa mereka sedang memproses cerita dengan lancar, dan baru akan menyadari proses ini ketika mereka mengalami kesulitan dalam memahami alur cerita atau menemukan hal yang tidak realistis.
2. *Attentional Focus* (Fokus Perhatian) Dimensi ini mendeskripsikan seberapa besar fokus audiens terhadap cerita yang disajikan dan seberapa jauh mereka dari distraksi (gangguan). Audiens yang benar-benar terhanyut ke dalam cerita tidak akan sadar bahwa perhatian mereka sedang terfokus secara intens. Mereka baru akan menyadari fokus mereka apabila pikiran mereka mulai mengembara (*mind wandering*) atau ketika mereka harus memusatkan kembali perhatian mereka karena adanya gangguan.
3. *Emotional Engagement* (Keterlibatan Emosional) Dimensi ini mengukur respons emosional yang dirasakan audiens terhadap karakter di dalam cerita. Keterlibatan ini mencakup dua hal: *empathy* (kemampuan merasakan emosi yang sama dengan yang dirasakan karakter) dan *sympathy* (perasaan peduli atau bersimpati terhadap kondisi karakter). Dimensi ini secara spesifik berpusat pada

komponen kebangkitan emosional (*emotional arousal*) dari keterlibatan sebuah narasi.

4. *Narrative Presence* (Kehadiran Naratif) Dimensi ini adalah sensasi di mana seseorang merasa telah meninggalkan dunia nyata dan masuk ke dalam dunia cerita (*story world*). Sensasi ini terdiri dari dua fenomena sekaligus: pertama, hilangnya kesadaran diri dan lingkungan sekitar akibat fokus yang terlalu intens (mirip dengan sensasi *flow*); dan kedua, sensasi memasuki ruang dan waktu alternatif yang merupakan ciri khas unik dari sebuah pengalaman naratif.

2.6 Dynamic Dialogue

Dynamic dialogue merupakan sebuah sistem percakapan dalam *video game* yang memungkinkan terjadinya interaksi secara lebih bebas dibandingkan dengan sistem dialog tradisional yang kaku. Melalui pendekatan ini, pemain diberikan kemampuan untuk memengaruhi pembentukan narasi secara langsung melalui input atau interaksi yang mereka lakukan. Implementasi dialog dinamis bertujuan untuk memberikan kebebasan yang lebih besar kepada pemain dalam membentuk arah cerita, sehingga pengalaman naratif yang dihasilkan menjadi lebih personal dan tidak terbatas pada opsi-opsi statis yang telah ditentukan sebelumnya [4].

2.7 Large Language Model (LLM)

Large Language Models (LLM) merupakan teknologi kecerdasan buatan berbasis *transformer architecture* yang dilatih menggunakan dataset teks berskala besar (mencapai miliaran hingga triliun token) untuk mengenali, memprediksi, dan menghasilkan teks yang menyerupai bahasa manusia. Pada tingkat paling mendasar, LLM beroperasi sebagai mesin prediksi teks (*text completion engines*). Model menerima masukan berupa rangkaian karakter atau kata (disebut *prompt*) dan memprosesnya untuk memberikan keluaran berupa kelanjutan teks yang secara statistik paling mungkin (*completion/response*) [13].

2.8 Prompt Engineering

Prompt engineering didefinisikan sebagai seni dan sains dalam merancang, menyusun, dan mengoptimalkan teks masukan (*prompt*) agar LLM dapat menghasilkan keluaran yang akurat, relevan, dan sesuai dengan kebutuhan pengguna. Karena LLM bekerja dengan cara mencocokkan pola yang pernah

dilihatnya selama masa pelatihan, efektivitas interaksi sangat bergantung pada bagaimana informasi dikemas ke dalam ruang dokumen model (*model domain*) [13].

Dalam praktiknya, penyusunan *prompt* yang ideal untuk mengarahkan perilaku model secara efektif melibatkan struktur anatomi yang matang. Berdasarkan metodologi perancangan instruksi, terdapat 4 komponen penting di dalam sebuah *prompt*:

1. *Introduction*: Bagian awal yang berfungsi menetapkan konteks utama, menetapkan peran/persona model, atau mendefinisikan jenis dokumen yang ingin dihasilkan (misalnya, laporan analitis atau percakapan konsultatif).
2. *Individual Prompt Elements*: Komponen-komponen spesifik yang berisi data statis (instruksi mendetail, contoh *few-shot*) maupun data dinamis (konteks tambahan dari hasil pencarian/RAG) yang dikumpulkan dan disajikan untuk memperkaya pemahaman model terhadap tugas.
3. *Refocus*: Strategi penempatan atau penekanan kembali pada bagian kritis di dalam *prompt*. Mengingat keterbatasan arsitektur LLM yang memproses teks secara sekuensial (unidireksional), model rentan kehilangan fokus pada instruksi yang tenggelam di tengah dokumen panjang, sehingga diperlukan elemen penegas (*refocus*) di bagian akhir sebelum model melakukan kompleks.
4. *Transition*: Tanda atau frasa struktural (seperti penggunaan format sintaksis khusus, *markdown*, atau karakter pembatas) yang memisahkan antara bagian instruksi, data konteks, dan area di mana model harus mulai menuliskan responsnya agar informasi tidak saling tumpang tindih.

2.8.2 System Prompt

System Prompt adalah instruksi tingkat tinggi (*meta-instruction*) yang diberikan kepada model untuk menetapkan aturan dasar, batasan perilaku, gaya bahasa, serta koridor keselamatan yang harus dipatuhi oleh AI selama sesi interaksi. Komponen ini berada pada lapisan sistem aplikasi dan biasanya tidak dapat diubah secara langsung oleh pengguna akhir. *System prompt* berfungsi sebagai "jangkar" yang memastikan model tetap konsisten pada perannya (misalnya, menjadi asisten

pemrograman yang objektif), meminimalkan risiko halusinasi, serta mencegah upaya manipulasi instruksi (*jailbreaking*) dari pihak luar.

2.8.3 User Prompt

User Prompt adalah masukan langsung, pertanyaan, atau perintah spesifik yang dimasukkan oleh pengguna manusia (atau dikirimkan secara dinamis oleh aplikasi) yang merepresentasikan masalah dunia nyata yang ingin diselesaikan saat itu juga. Berbeda dengan *system prompt* yang bersifat struktural dan permanen, *user prompt* bersifat ad-hoc, kontekstual, dan bervariasi tergantung pada kebutuhan interaksi spesifik pengguna. Aplikasi kemudian bertindak sebagai lapisan transformasi yang menggabungkan *user prompt* ini dengan *system prompt* dan konteks dinamis lainnya menjadi satu kesatuan dokumen utuh sebelum dieksekusi oleh model.

2.9 LLM as a Judge

LLM as a Judge merupakan paradigma evaluasi di mana Large Language Model digunakan sebagai evaluator otomatis untuk menilai kualitas keluaran sistem Natural Language Generation atau respons model lain, dengan mengandalkan kemampuan pemahaman bahasa alami dan penalaran model tersebut untuk memberikan penilaian terstruktur berdasarkan kriteria yang telah ditentukan [25]. Pendekatan ini menawarkan alternatif evaluasi yang lebih scalable dan fleksibel dibandingkan evaluasi manusia, serta dapat disesuaikan dengan berbagai dimensi penilaian melalui desain prompt yang eksplisit [25].

2.10 G-Eval

G-Eval merupakan kerangka evaluasi berbasis Large Language Model yang digunakan untuk menilai kualitas keluaran Natural Language Generation (NLG). Kerangka ini memanfaatkan kemampuan LLM sebagai evaluator dengan menggabungkan chain-of-thought dan paradigma form-filling agar proses penilaian dapat dilakukan secara lebih sistematis dan terstruktur [23]. Dalam penerapannya, G-Eval dirancang untuk meningkatkan kesesuaian penilaian LLM dengan penilaian manusia pada tugas evaluasi teks, sehingga evaluasi tidak hanya dilakukan berdasarkan impresi umum model, melainkan melalui kriteria dan prosedur penilaian yang eksplisit. Komponen utama G-Eval adalah sebagai berikut:

1. Task introduction, yaitu bagian yang menjelaskan jenis keluaran yang akan dinilai serta tujuan dari proses evaluasi tersebut
2. Evaluation criteria, yaitu dasar penilaian yang digunakan untuk menilai aspek-aspek tertentu dari teks yang dihasilkan, seperti koherensi, relevansi, atau kualitas bahasa
3. Evaluation steps, yaitu langkah-langkah penalaran bertahap yang harus diikuti model sebelum memberikan skor akhir, sehingga proses penilaian menjadi lebih eksplisit dan dapat ditelusuri
4. Scoring form, yaitu format keluaran penilaian yang digunakan untuk membatasi bentuk jawaban model agar hasil evaluasi dapat direpresentasikan secara konsisten, seperti skor numerik atau label kualitas tertentu. Pada contoh penerapan G-Eval untuk tugas evaluasi NLG, dimensi coherence dinilai menggunakan skala 1–5, dengan instruksi eksplisit bahwa 1 merupakan skor terendah dan 5 merupakan skor tertinggi berdasarkan kriteria evaluasi yang telah ditetapkan [23].

2.11 Uncertainty

Uncertainty merupakan tingkat ketidakpastian model dalam menghasilkan prediksi atau keluaran, yang penting untuk memahami reliabilitas dan kepercayaan terhadap hasil yang dihasilkan oleh Large Language Model. Dalam konteks LLM, pengukuran uncertainty memungkinkan identifikasi situasi di mana model memiliki keyakinan rendah terhadap keluarannya, sehingga dapat digunakan sebagai sinyal untuk melakukan intervensi seperti regenerasi respons, permintaan klarifikasi, atau eskalasi ke evaluator alternatif [24]. Pengukuran uncertainty pada LLM dapat dikategorikan berdasarkan sumber ketidakpastian yang diukur. Jenis-jenis uncertainty yang relevan dalam evaluasi LLM adalah sebagai berikut:

1. Aleatoric uncertainty, yaitu ketidakpastian yang muncul dari variabilitas inheren dalam data atau tugas itu sendiri, seperti ambiguitas bahasa alami, noise dalam data, atau ketidaklengkapan informasi input, yang bersifat irreducible dan tidak dapat dikurangi meskipun arsitektur model ditingkatkan atau data pelatihan ditambah [24];

2. Epistemic uncertainty, yaitu ketidakpastian yang berasal dari keterbatasan pengetahuan model atau kurangnya data pelatihan pada domain tertentu, yang bersifat reducible dan dapat dikurangi melalui peningkatan arsitektur model, penambahan data pelatihan yang relevan, atau fine-tuning pada domain spesifik [24];
3. Distributional uncertainty, yaitu ketidakpastian yang muncul ketika data input berada di luar distribusi data pelatihan model (out-of-distribution), sehingga model menghadapi situasi yang tidak familiar dan cenderung menghasilkan prediksi dengan keyakinan rendah karena kurangnya contoh serupa dalam proses pembelajaran [24].

2.12 AI Agent

Secara fundamental, agen didefinisikan sebagai sistem otonom yang dapat melakukan berbagai tugas, membuat keputusan, serta berinteraksi dengan pengguna atau lingkungannya. Agen otonom sejati adalah sistem perangkat lunak cerdas yang dirancang untuk secara mandiri menganalisis data, menafsirkan lingkungan mereka, dan membuat keputusan yang didorong oleh konteks (*context-driven reasoning*). Berbeda dengan perangkat lunak tradisional, agen mampu beradaptasi dengan skenario yang berubah dan melakukan tindakan yang cangguh dengan pengawasan manusia yang minimal [14].

2.13 Multi Agent System

Sistem *multi-agent* adalah arsitektur di mana beberapa *agent* bekerja sama untuk mencapai sebuah tujuan bersama. Pendekatan ini sangat menguntungkan ketika tugas yang dihadapi sangat kompleks dan membutuhkan beragam alat, pemrosesan paralel, atau kemampuan beradaptasi pada lingkungan yang dinamis. Manfaat utama dari sistem multi-agen adalah adanya spesialisasi: setiap agen dapat diberikan peran atau bidang keahlian tertentu. Pembagian kerja ini memungkinkan sistem untuk menangani tugas dengan lebih efisien karena agen berfokus pada pekerjaan spesifiknya tanpa ada upaya yang tumpang tindih [14].

2.11.1 Autonomous

Pendekatan otonom merujuk pada situasi di mana sistem dirancang untuk beroperasi secara asinkron; mampu mengelola banyak tugas secara paralel,

beradaptasi dengan informasi baru, dan memprioritaskan tindakan secara dinamis berdasarkan kondisi yang berubah-ubah. Pendekatan otonom sangat tepat digunakan ketika input yang masuk tidak terstruktur, baru, atau memiliki variabilitas yang tinggi, serta ketika sistem dituntut untuk melakukan perencanaan taktis dinamis dalam banyak langkah (*multistep planning*) atau pembelajaran berkelanjutan dari umpan balik.

2.11.2 Pre Define

Pendekatan deterministik atau *pre-define* adalah pendekatan logika yang diekspresikan sebagai sekumpulan langkah atau cabang keputusan yang terbatas dan sudah diketahui sejak awal. Jika input sepenuhnya dapat diprediksi dan setiap kemungkinan keluaran (output) dapat dideskripsikan di awal, menggunakan baris kode prosedural yang sederhana sering kali lebih cepat, lebih murah, dan jauh lebih mudah diuji dibandingkan menggunakan model AI. Pendekatan ini sangat cocok ketika pembuat sistem perlu memiliki kontrol yang ketat dan dapat diaudit secara pasti pada setiap cabang pengambilan keputusan.

2.11.3 LangGraph

LangGraph merupakan framework orkestrasi yang digunakan untuk membangun alur kerja aplikasi berbasis Large Language Model dalam bentuk graf. Framework ini memungkinkan proses yang melibatkan beberapa agen direpresentasikan sebagai tahapan-tahapan yang saling terhubung. LangGraph mendukung pengelolaan data selama proses berlangsung, urutan eksekusi, percabangan berdasarkan kondisi, dan pengulangan menuju tahapan sebelumnya [14]. Dalam sistem multi-agent, setiap agen dapat memiliki tanggung jawab yang berbeda. Namun, keluaran suatu agen dapat menjadi masukan bagi agen berikutnya. LangGraph mengatur hubungan tersebut melalui tiga komponen utama, yaitu state, node, dan edge [14]. Ketiga komponen tersebut dijelaskan sebagai berikut:

1. State: merupakan struktur data yang menyimpan informasi selama graph dijalankan. State menjadi media pertukaran data antarnode sehingga keluaran dari suatu proses dapat digunakan oleh proses selanjutnya. Isi state ditentukan berdasarkan informasi yang perlu dipertahankan selama alur kerja berlangsung.

2. Node: merupakan unit proses atau fungsi yang dijalankan di dalam graph. Setiap node menerima state sebagai masukan, menjalankan tugas tertentu, kemudian mengembalikan data untuk memperbarui state. Node dapat berisi pemanggilan LLM, validasi data, transformasi informasi, maupun proses lainnya.
3. Edge: merupakan hubungan yang menentukan perpindahan proses dari satu node menuju node lain. Edge mengatur urutan eksekusi graph. Dalam LangGraph, perpindahan tersebut dapat berupa edge langsung dan edge bersyarat.

2.11.4 Actor-Critic Agent

Pendekatan *Actor-Critic* dalam sistem agenis adalah bentuk iterasi ringan yang didorong oleh evaluasi pengujian (*evaluation-driven iteration*). Dalam pengaturan ini:

1. Aktor (Actor) bertanggung jawab untuk menghasilkan kandidat keluaran—seperti jawaban, rencana, atau serangkaian tindakan.
2. Kritikus (Critic) bertindak sebagai gerbang kualitas (*quality gate*) yang menerima atau menolak keluaran dari aktor tersebut berdasarkan pada rubrik atau panduan penilaian yang telah ditentukan sebelumnya.

Proses ini bekerja secara sederhana: aktor akan terus memproduksi draf kandidat sampai kritikus menentukan bahwa keluaran tersebut telah memenuhi ambang batas kualitas yang diinginkan, bertindak sebagai filter yang kuat ketika sebuah tugas bersifat generatif.

2.14 Evaluasi Kualitas Dialog

Evaluasi dialog merupakan proses penilaian terhadap kualitas respons percakapan berdasarkan sejumlah aspek yang terdefinisi. Kualitas dialog tidak cukup dinilai menggunakan satu ukuran umum, karena dialog memiliki beberapa karakteristik, seperti relevansi terhadap konteks, ketepatan makna, kelancaran bahasa, dan kemampuan mempertahankan keterlibatan lawan bicara. Mehri dan Eskenazi memperkenalkan Fine-grained Evaluation of Dialog (FED) sebagai kerangka evaluasi dialog yang menilai kualitas percakapan melalui sejumlah kualitas berbutir halus (*fine-grained dialog qualities*) [19]. FED membedakan

evaluasi pada dua level, yaitu turn-level dan dialog-level. Evaluasi turn-level digunakan untuk menilai kualitas satu respons terhadap konteks percakapan sebelumnya, sedangkan evaluasi dialog-level digunakan untuk menilai kualitas percakapan secara keseluruhan [19].

Pada level turn-level, FED mencakup beberapa kualitas, yaitu interesting, engaging, specific, relevant, correct, semantically appropriate, understandable, dan fluent [19]. Dalam anotasi manusia pada FED, kualitas-kualitas tersebut dinilai menggunakan opsi No, Somewhat, Yes, dan N/A, kecuali pada dimensi understandable yang tidak menggunakan opsi Somewhat. Sementara itu, penilaian overall impression menggunakan skala Likert lima poin [19]. Berdasarkan kebutuhan penelitian ini, rubrik evaluasi dialog disusun dengan mengadopsi enam dimensi dari kualitas turn-level FED, yaitu relevance, semantic appropriateness, specificity, understandability, fluency, dan engagingness. Selain itu, penelitian ini menambahkan empat dimensi yang disesuaikan dengan kebutuhan dialog NPC dalam game, yaitu persona consistency, emotional alignment, style compliance, dan goal fulfillment. Penambahan dimensi tersebut diperlukan karena dialog NPC tidak hanya dinilai sebagai respons percakapan umum, tetapi juga sebagai bagian dari karakter, narasi, dan tujuan interaksi dalam game. Kesepuluh dimensi tersebut dijelaskan sebagai berikut:

1. Relevance: merupakan dimensi yang menilai sejauh mana respons berkaitan dengan konteks percakapan yang sedang berlangsung. Dalam FED, dimensi ini direpresentasikan melalui pertanyaan apakah respons relevan terhadap percakapan [19]. Respons yang relevan harus menjawab atau merespons situasi percakapan sebelumnya, bukan berpindah ke topik yang tidak berkaitan. Pada dialog NPC, dimensi ini penting karena respons yang tidak relevan dapat membuat percakapan terasa terputus dan mengurangi keterhubungan antara aksi pemain dengan respons karakter [19].
2. Semantic appropriateness: merupakan dimensi yang menilai apakah makna respons sesuai dengan konteks tuturan. FED menggunakan dimensi ini untuk menilai apakah respons secara semantik tepat terhadap percakapan [19]. Dimensi ini berbeda dari kelancaran bahasa karena respons dapat saja ditulis

dengan tata bahasa yang baik, tetapi maknanya tidak sesuai dengan situasi. Dalam evaluasi dialog, semantic appropriateness digunakan untuk memastikan bahwa isi respons masuk akal dan tidak menimbulkan ketidaksesuaian makna terhadap konteks yang diberikan [19].

3. **Specificity:** merupakan dimensi yang menilai apakah respons bersifat spesifik terhadap percakapan atau terlalu generik. Dalam FED, dimensi ini dirumuskan melalui pertanyaan apakah respons bersifat umum atau spesifik terhadap percakapan [19]. Respons yang terlalu generik dapat digunakan pada berbagai situasi tanpa memperhatikan konteks tertentu, sehingga kualitasnya lebih rendah. Pada dialog NPC, specificity dibutuhkan agar respons karakter terasa menanggapi kondisi yang benar-benar terjadi, bukan hanya menghasilkan jawaban umum yang tidak memiliki keterikatan dengan situasi permainan [19].
4. **Understandability:** merupakan dimensi yang menilai apakah respons mudah dipahami oleh pembaca atau lawan bicara. FED memasukkan understandability sebagai salah satu kualitas turn-level untuk menilai apakah respons dapat dipahami [19]. Dimensi ini berkaitan dengan kejelasan struktur kalimat, keterbacaan, dan tidak adanya ambiguitas yang menghambat pemahaman. Dalam konteks game, respons NPC yang sulit dipahami dapat mengganggu pemain dalam menangkap informasi, suasana, maupun arah percakapan [19].
5. **Fluency:** merupakan dimensi yang menilai kelancaran respons dari sisi kebahasaan. Dalam FED, fluency digunakan untuk menilai apakah respons ditulis secara fasih atau lancar [19]. Dimensi ini mencakup kewajaran susunan kalimat, tata bahasa, dan aliran bahasa yang natural. Namun, fluency tidak sama dengan kualitas isi. Sebuah respons dapat fasih secara bahasa, tetapi tetap bernilai rendah apabila tidak relevan, tidak tepat secara semantik, atau tidak sesuai dengan karakter [19].
6. **Engagingness:** merupakan dimensi yang menilai kemampuan respons dalam mempertahankan ketertarikan dan keterlibatan lawan bicara. FED menggunakan dimensi ini untuk menilai apakah respons bersifat engaging [19]. Dalam dialog interaktif, respons yang engaging tidak hanya menjawab konteks, tetapi juga membuat percakapan tetap hidup dan layak dilanjutkan. Pada dialog

NPC, dimensi ini penting karena percakapan yang datar dapat mengurangi minat pemain terhadap karakter maupun situasi naratif yang sedang berlangsung [19].

7. **Persona consistency:** merupakan dimensi yang menilai kesesuaian respons dengan persona atau karakteristik tokoh. Dimensi ini ditambahkan karena dialog NPC perlu mempertahankan identitas karakter secara konsisten. Li dkk. menunjukkan bahwa persona dapat digunakan untuk membangun model percakapan yang menghasilkan respons lebih konsisten terhadap karakter tertentu [20]. Selain itu, Zhang dkk. melalui Persona-Chat menekankan pentingnya personalisasi dialog berdasarkan profil persona agar agen percakapan dapat mempertahankan identitas dan informasi persona selama interaksi [21]. Dalam penelitian ini, persona consistency digunakan untuk menilai apakah respons NPC sesuai dengan profil kepribadian, identitas, dan karakter yang telah ditentukan [20], [21].
8. **Emotional alignment:** merupakan dimensi yang menilai kesesuaian emosi dalam respons dengan keadaan percakapan. Dimensi ini tidak berasal langsung dari daftar kualitas turn-level FED, tetapi ditambahkan sebagai kebutuhan domain game karena dialog NPC perlu menampilkan reaksi emosional yang sesuai dengan situasi interaksi. Pada dialog berbasis karakter, kesesuaian emosi berkaitan dengan konsistensi persona, karena respons karakter seharusnya mencerminkan sikap, keadaan, dan kecenderungan karakter yang telah ditentukan [20], [21]. Dalam penelitian ini, emotional alignment digunakan untuk menilai apakah emosi yang muncul dalam dialog, seperti kecewa, marah, lega, atau senang, sesuai dengan state dialog yang sedang berlangsung.
9. **Style compliance:** merupakan dimensi yang menilai kepatuhan respons terhadap gaya bahasa yang telah ditentukan. Dimensi ini ditambahkan karena dialog NPC tidak hanya perlu menyampaikan isi yang benar, tetapi juga harus terdengar sesuai dengan karakter yang berbicara. Penelitian dialog berbasis persona menunjukkan bahwa karakteristik persona memengaruhi bagaimana respons seharusnya dibentuk agar tetap konsisten dengan identitas penutur [20], [21]. Dalam konteks game, gaya bahasa dapat ditentukan oleh usia, kepribadian,

register bahasa, atau desain karakter. Oleh karena itu, style compliance digunakan untuk memastikan bahwa respons NPC sesuai dengan gaya tutur yang telah ditetapkan [20], [21].

2.15 OCEAN

Model kepribadian *Big Five* atau OCEAN merupakan kerangka kerja psikologis yang mengklasifikasikan kepribadian manusia ke dalam lima dimensi utama, yaitu *Openness* (keterbukaan), *Conscientiousness* (kesadaran), *Extraversion* (ekstraversi), *Agreeableness* (keramahan), dan *Neuroticism* (neurotisisme). Dalam pengembangan kecerdasan buatan untuk *video game*, model ini digunakan sebagai parameter untuk memberikan bias pada perilaku dan gaya bahasa agen agar memiliki karakter yang khas dan konsisten. Dengan menerapkan variabel OCEAN pada agen dialog, sistem dapat menghasilkan respons yang lebih personal dan bervariasi secara otomatis, sehingga meningkatkan realisme interaksi antara pemain dan NPC dalam sebuah narasi interaktif [5]. Berikut ini adalah 5 dimensi dari OCEAN model:

1. *Openness to Experience* (Keterbukaan terhadap Pengalaman) Dimensi ini mendeskripsikan tingkat kreativitas dan variasi topik yang diminati oleh seseorang. Dalam konteks agen permainan, sifat keterbukaan memengaruhi keputusan agen ketika ia dihadapkan pada tindakan yang baru pertama kali dilakukannya (mengubah kalkulasi biaya atau *cost* terhadap aksi baru tersebut).
2. *Conscientiousness* (Kehati-hatian / Kesadaran) Dimensi ini mendeskripsikan tingkat organisasi dan motivasi individu dalam mencapai tujuan-tujuannya. Pada implementasi perilaku agen NPC, sifat ini memengaruhi manajemen tujuan (*goals*) agen, seperti apakah agen akan menghapus tujuan saat ini demi tujuan baru, atau justru menyusunnya dengan rapi ke dalam antrean tugasnya.
3. *Extraversion* (Ekstraversi) Dimensi ini mengukur tingkat interaksi dan hubungan antarpribadi (interpersonal) yang dimiliki seseorang. Dalam sistem agen *multi-agent*, nilai ekstraversi yang tinggi akan menurunkan hambatan (menurunkan *cost* keputusan) bagi agen untuk melakukan tindakan-tindakan yang membutuhkan komunikasi dan interaksi dengan NPC atau karakter lain.

4. *Agreeableness* (Keramahan / Kesesuaian) Dimensi ini mendefinisikan seberapa mampu seorang individu untuk beradaptasi dengan orang lain. Untuk agen di dalam *game*, tingkat kesesuaian ini secara langsung berdampak pada kecenderungan agen dalam memilih tindakan yang membutuhkan persetujuan (*consent*) dari pihak lain.
5. *Neuroticism* (Neurotisme) Dimensi ini mengukur tingkat stabilitas emosional dari seorang individu. Dalam sistem emosi agen dinamis, tingkat neurotisme bertindak sebagai pengatur ambang batas transisi (*transition thresholds*) emosional. Sifat neurotisme yang tinggi merepresentasikan ketidakstabilan emosional, sehingga agen tersebut akan lebih cepat dan sensitif dalam mengubah status emosinya (seperti mudah marah, takut, atau gembira) saat menerima rangsangan peristiwa.

2.16 Unity Engine

Unity merupakan sebuah *game engine* multiplatform yang menyediakan lingkungan pengembangan terintegrasi (*Integrated Development Environment*) untuk membangun objek dua dimensi (2D) maupun tiga dimensi (3D). Dalam pengembangan *Visual Novel* dan sistem dialog dinamis, Unity berfungsi sebagai antarmuka utama (*frontend*) yang menangani logika permainan, manajemen aset visual, serta interaksi pengguna. Keunggulan utama Unity terletak pada arsitekturnya yang modular dan fleksibel, yang memungkinkan pengembang untuk menghubungkan logika permainan internal dengan modul kecerdasan buatan eksternal melalui protokol komunikasi data, sehingga sangat ideal untuk mengimplementasikan sistem narasi yang kompleks [15].

2.17 Python

Python adalah bahasa pemrograman tingkat tinggi yang bersifat interpretatif dan berorientasi objek, yang secara luas digunakan dalam pengembangan kecerdasan buatan (*Artificial Intelligence*) dan pengolahan data. Dalam arsitektur sistem dialog dinamis, Python berperan sebagai *backend* utama yang menangani pemrosesan logika *Large Language Models* (LLM) serta manajemen *Multi-Agent System*. Fleksibilitas Python dalam menangani pustaka (*library*) pembelajaran mesin dan komunikasi data melalui API memungkinkannya untuk berfungsi

sebagai jembatan komputasi yang efisien, di mana data input dari pemain diproses di sisi server sebelum dikembalikan ke dalam lingkungan *game engine* [16].

2.18 *Fast API*

2.19 *Client-Server*

2.20 *Class*

BAB 3

ANALISIS DAN PERANCANGAN

3.1 Analisis Sistem

Analisis sistem bertujuan untuk mengidentifikasi masalah yang ada pada *game* “Tea’n Brew” serta menentukan kebutuhan untuk pembangunan sistem. Diperlukan pemahaman yang mendalam terhadap sistem berjalan untuk melakukan tahapan *domain understanding* hingga pengujian.

3.1.1 Domain Understanding

Game “Tea’n Brew” merupakan sebuah *game* dengan genre *visual novel* yang memiliki konsep dimana pemain akan berperan sebagai penjaga *cafe* yang akan melayani *customer* serta mendengarkan curhatan mereka. *Game* “Tea’n Brew” sendiri merupakan *game endless* dimana pemain dapat terus bermain untuk mendapat *score* setinggi-tingginya dengan tantangan pada *game* ini adalah pemain harus memenuhi *target score* yang diberikan setiap harinya, bila pemain gagal memenuhi *target score* maka *game* akan berakhir atau *game over*. Terdapat beberapa skenario yang ada pada *game* “Tea’n Brew” yang dapat dilihat pada **Tabel 3.1:**

Tabel 3.1 List skenario *game* “Tea’n Brew”

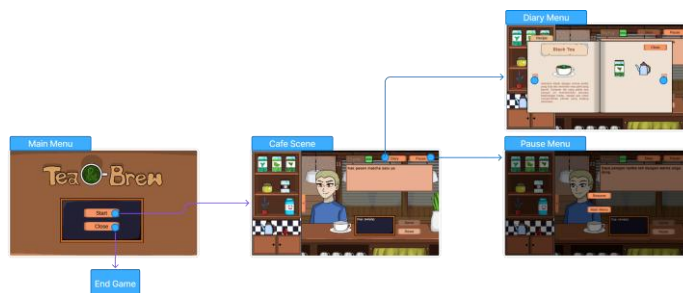
No	Skenario	Penjelasan
1	<i>Menu</i>	Skenario dimana pemain membuka beberapa menu yang ada pada <i>game</i> seperti main menu, pause menu, dan diary menu.
2	<i>Customer Came</i>	Skenario dimana pemain memulai permainan
3	<i>Make Tea</i>	Skenario pemain saat <i>customer</i> memesan pesanan, pemain dapat membuatkan pesanannya
4	<i>Venting Session</i>	Skenario saat pemain berhasil membuat pesanan <i>customer</i> dengan tepat
5	<i>Daily Cycle</i>	Skenario saat pemain memenuhi <i>target score</i> yang diberikan per harinya
6	<i>Game Over</i>	Skenario saat pemain gagal memenuhi <i>target score</i> per harinya

Berikut ini adalah penjelasan setiap skenario yang ada pada *game*:

1. Skenario *Menu*

Skenario *main menu* merupakan skenario dimana pemain dapat membuka beberapa *menu* yang ada pada *game* yaitu:

1. *main menu*: Merupakan menu utama yang muncul saat awal permainan dimulai
 2. *pause menu*: Digunakan untuk menjeda *game* dan memunculkan *menu pause* yang berisi tombol *continue* dan *main menu*
 3. *diary menu*: Digunakan untuk membuka *menu* yang berisi resep minuman
- User flow* dari skenario menu ini dapat dilihat pada **Gambar 3.1**.



Gambar 3.1 *User flow* skenario menu

Untuk konten minuman yang dapat dibuat pemain yang terdapat di *dary menu* dapat dilihat pada **Tabel 3.2**.

Tabel 3.2 Daftar Resep Minuman

Minuman	Deskripsi
<i>Black Tea</i>	Seduhan klasik dengan aroma earthy yang kuat dan sentuhan rasa pahit yang bersih. Karakter teh yang pekat dan hangat ini memberikan sensasi ketenangan instan, sangat pas untuk menjernihkan pikiran yang sedang semrawut.
<i>Green Tea</i>	Minuman ringan dengan aroma daun segar yang menenangkan dan sentuhan rasa grassy yang khas. Seduhan berwarna hijau bening ini memberikan sensasi rileks yang lembut, sangat cocok untuk mendinginkan kepala yang sedang panas.
<i>Mint Tea</i>	Perpaduan aromatik yang memberikan sensasi dingin semilir di tengah kehangatan air seduhan. Rasa mentolnya yang tajam sangat ampuh untuk melegakan pernapasan dan memberikan efek refreshing yang instan bagi pikiran yang lelah.
<i>Lavender Tea</i>	Seduhan berwarna ungu cantik yang memikat mata dengan aroma floral yang sangat menyenangkan. Sentuhan manis dari madu menyeimbangkan rasa unik lavender, menciptakan sensasi relaksasi mendalam yang mampu menghapus segala ketegangan hari ini.
<i>Jasmine Tea</i>	Perpaduan klasik yang memikat, di mana ketegasan teh hitam bertemu dengan kelembutan aroma bunga melati. Rasa bold yang dibalut keharuman floral ini menciptakan sensasi elegan di setiap sesapan, memberikan ketenangan sekaligus semangat baru bagi jiwa yang sedang suntuk.
<i>Milk Tea</i>	Minuman yang lembut dan creamy, memadukan kekuatan teh hitam dengan kehalusan susu serta manisnya madu yang alami. Teksturnya yang kental

Minuman	Deskripsi
	memberikan kenyamanan luar biasa, seolah memberikan pelukan hangat bagi siapa pun yang sedang membutuhkan ketenangan ekstra.
<i>Mint Milk Tea</i>	Perpaduan unik antara rasa manis yang lembut dari susu dan madu dengan sensasi dingin yang menyegarkan dari daun mint. Teksturnya yang creamy namun memberikan efek sejuk di tenggorokan menjadikannya minuman sempurna untuk meredakan amarah atau sekadar menyegarkan pikiran.
<i>Matcha Latte</i>	Minuman dengan tekstur creamy yang menonjolkan perpaduan rasa bittersweet dari teh hijau pekat dan kelembutan susu. Warna hijaunya yang teduh serta aroma khas pucuk daun segar memberikan ketenangan maksimal, sangat pas untuk menstabilkan suasana hati yang sedang kacau.
<i>Honey Tea</i>	Minuman klasik yang menenangkan, memadukan ketegasan teh hitam dengan manisnya madu yang lembut. Rasa manis alaminya menyelimuti rasa sepat teh dengan sempurna, menciptakan kehangatan instan yang mampu melegakan tenggorokan sekaligus menghibur perasaan yang sedang sedih.
<i>Golden Jasmine Tea</i>	Minuman elegan yang memadukan keharuman bunga melati dengan kelembutan susu dan sentuhan manis madu. Teksturnya yang halus dan aromanya yang menenangkan menciptakan sensasi mewah yang menyejukkan, sangat cocok untuk meredakan ketegangan setelah hari yang panjang.
<i>Apple Tea</i>	Perpaduan harmonis antara rasa sepat teh hitam, segarnya sari apel, dan manisnya madu yang lembut. Aromanya yang manis dan menenangkan memberikan kehangatan ekstra bagi tubuh, sangat cocok dinikmati saat kamu ingin merasa lebih rileks dan nyaman.
<i>Apple Green Tea</i>	Seduhan jernih yang memadukan kesegaran teh hijau dan sari apel dengan sentuhan manis madu yang lembut. Rasanya yang ringan namun kaya akan nutrisi memberikan sensasi pemulihan yang menenangkan, sangat cocok untuk mengembalikan energi tanpa rasa berat di lidah.

2. Skenario *Customer Came*

Skenario ini dimulai dari *cafe scene* dimana *customer* akan datang dan memesan minuman dengan memunculkan dialog *customer came*. Berikut ini adalah user flow dari skenario *customer came* yang dapat dilihat pada **Gambar 3.2**.



Gambae 3.2 User flow skenario *customer came*

Pada skenario ini pula *customer* yang datang akan melalui sebuah mekanik *randomize customer*. Mekanik ini memungkinkan mengacak *profile customer* yang berdasarkan rentang usia beserta *sprite*, nama, dan dialog yang dikeluarkan

customer. Berikut ini adalah gambaran alur atau tahapan mekanik *randomize customer* yang dapat dilihat pada **Gambar 3.3**.



Gambar 3.3 Mekanik *randomize customer*

Terdapat beberapa *profile customer* yang diacak melalui mekanik *randomize customer* yang dapat dilihat pada **Tabel 3.3**.

Tabel 3.3 List *Profile Customer* di Game

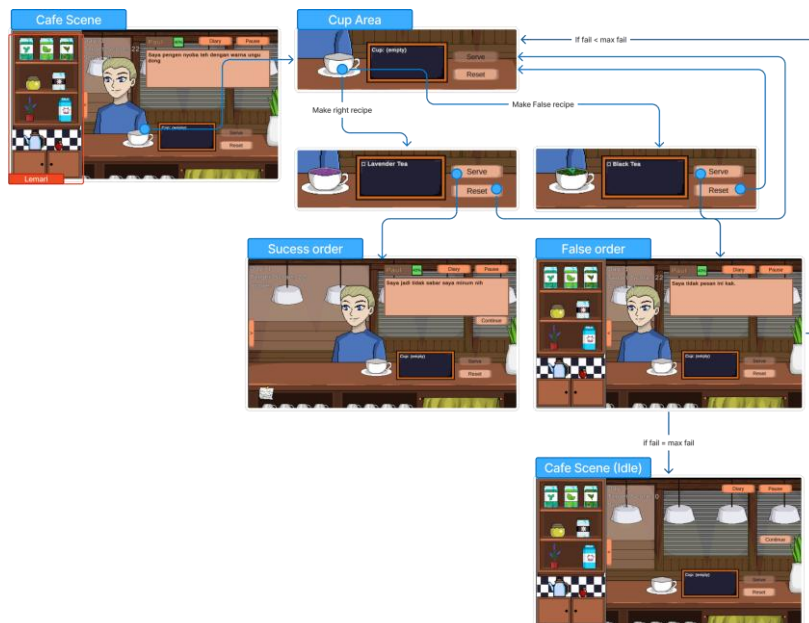
No	<i>Profile Customer</i>
1	<i>Teen Male</i> (Remaja Pria)
2	<i>Teen Female</i> (Remaja Wanita)
3	<i>Adult Male</i> (Dewasa Pria)
4	<i>Adult Female</i> (Dewasa Wanita)
5	<i>Old Male</i> (Orang Tua Pria)
6	<i>Old Female</i> (Orang Tua Wanita)

3. Skenario *Make Tea*

Skenario *make tea* merupakan skenario dimana pemain dapat membuatkan pesanan pada *customer*. Berikut ini langkah-langkah yang ada pada skenario ini:

- 1) Pemain mengambil bahan-bahan minuman dari lemari sesuai dengan resep yang ada pada *Diary Menu*.
- 2) Pemain meracik bahan-bahan tersebut menjadi sebuah minuman.
- 3) Pemain menghidangkan minuman yang telah dibuat kepada *customer*.
- 4) Jika minuman benar: *Customer* merasa puas dan sistem akan melanjutkan ke Skenario *Curhat Session*.
- 5) Jika minuman salah (belum mencapai batas *max_fail*): *Customer* memberitahu bahwa pesanan salah, dan pemain dapat mencoba membuat ulang.
- 6) Jika minuman salah (mencapai *max_fail*): *Customer* marah dan pergi dari *cafe*, lalu pemain mendapatkan pengurangan *score* sebesar -10.

Berikut ini adalah *user flow* dari skenario *make tea* yang dapat dilihat pada **Gambar 3.4**.



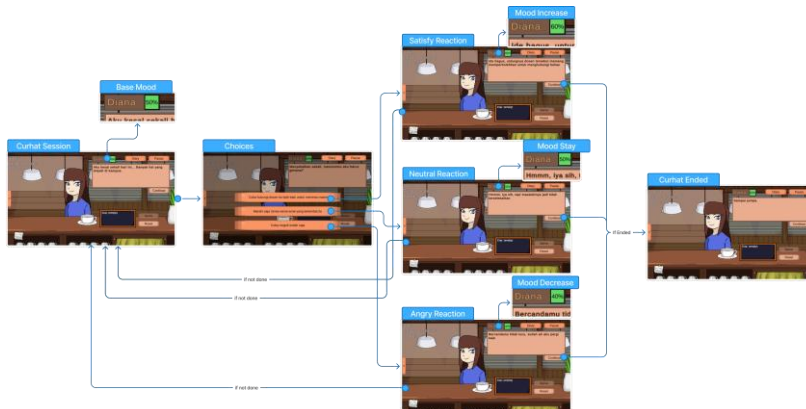
Gambar 3.4 User flow skenario make tea

4. Skenario Venting Session

Skenario *venting session* merupakan skenario yang akan muncul jika pemain berhasil membuat pesanan *customer* dengan tepat. Berikut ini langkah-langkah pada skenario ini:

- 1) *Customer* mulai menceritakan masalah atau keluhan kesahnya kepada pemain.
- 2) Pemain diberikan 3 pilihan jawaban atau reaksi terhadap curhatan tersebut.
- 3) Pemain memilih salah satu dari pilihan jawaban yang tersedia.
- 4) Sistem melakukan penyesuaian mood *customer* berdasarkan jawaban pemain:
 - a. Jawaban positif: *Mood* naik 10%.
 - b. Jawaban negatif: *Mood* turun 10%.
 - c. Jawaban netral: *Mood* tetap.
- 5) Bila sesi curhat masih berlangsung, maka kembali ke langkah 1
- 6) Bila sesi curhat berakhir maka pemain mendapatkan *score* yang besarnya dipengaruhi oleh persentase *mood* akhir *customer* tersebut

Berikut ini adalah *user flow* dari skenario *venting session* yang dapat dilihat pada **Gambar 3.5**.



Gambar 3.5 *User flow* skenario *venting session*

5. Skenario *Daily Cycle*

Skenario *daily cycle* akan muncul bila pemain berhasil memenuhi target *score* harian. Berikut ini adalah langkah-langkah dari skenario ini:

- 1) Pemain berhasil mencapai atau melampaui target *score* kumulatif yang diberikan pada hari tersebut
- 2) Sistem menampilkan status hari telah berakhir.
- 3) Pemain menekan tombol untuk memulai hari baru
- 4) Sistem memperbarui target *score* yang lebih tinggi untuk hari berikutnya.
- 5) Pemain kembali ke *Cafe Scene* untuk melayani *customer* selanjutnya.

Berikut ini *user flow* dari skenario *daily cycle* yang dapat dilihat pada **Gambar 3.6**.



Gambar 3.6 *User flow* skenario *daily cycle*

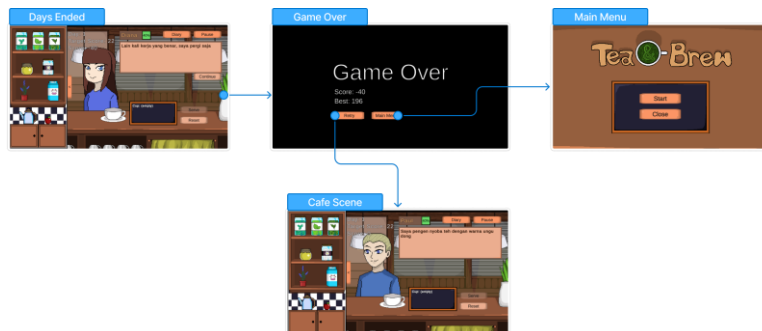
6. Skenario *Game Over*

Skenario ini akan muncul bila pemain gagal mencapai target *score* yang ditentukan, Berikut ini adalah langkah-langkah dari skenario ini:

- 1) Hari berakhir namun *score* yang dikumpulkan pemain tidak mencapai target *score* minimal yang ditentukan.
- 2) Sistem menghentikan permainan dan memunculkan layar "*Game Over*".
- 3) Layar menampilkan *score* akhir yang didapat dan *score* terbaik (*Best Score*) pemain.
- 4) Pemain memilih tombol "*Restart*" untuk mengulang permainan dari awal atau "*Main Menu*" untuk kembali ke *main menu*.

Berikut ini adalah user flow dari skenario *game over* yang dapat dilihat pada

Gambar 3.7.



Gambar 3.7 User flow skenario game over

Game "*Tea'n Brew*" sudah dirilis di platform itch.io pada tanggal 2 Januari 2026. Setelah game dirilis, ternyata ditemukan beberapa masalah yaitu *rating game* yang kurang baik yaitu di 3.06 dan beberapa *review* dari pemain, berikut ini adalah *review* dari pemain yang dapat dilihat pada **Lampiran A**.

Berdasarkan *review* pemain setelah dianalisa, 9 dari 18 atau sekitar 50% dari total orang yang *review game* ini mengatakan kalau dialog pada game terasa monoton dan berulang-ulang. Hal ini bisa disimpulkan bahwa terdapat masalah pada game tersebut terutama pada skenario *customer came*. Untuk memastikan permasalahan game lebih dalam, kuisisioner GEQ disebarakan kepada pemain. Tujuan penyebaran kuisisioner GEQ sendiri dilakukan untuk mencari dimensi bagian mana yang menjadi masalah utama yang dihadapi game "*Tea'n Brew*" ini. Berikut ini tabel hasil kuisisioner GEQ yang dapat dilihat pada **Tabel 3.5**.

Tabel 3.5 Hasil kuisisioner GEQ

Dimensi	Presentase	Kategori
<i>Immersion</i>	52.38%	Sedang
<i>Flow</i>	62.76%	Tinggi
<i>Competence</i>	81.43%	Sangat Tinggi
<i>Tension</i>	41.43%	Sedang
<i>Positive Affect</i>	67.24%	Tinggi

Berdasarkan isi tabel dapat dilihat bahwa dimensi yang seharusnya menjadi keunggulan dari game “*Tea’n Brew*” yaitu *Immersion* hanya berada di kategori sedang dengan presentase 52.38% [11]. Dimensi *Immersion* pada game “*Tea’n Brew*” mencakup keselarasan *profile customer* dengan dialog serta suasana dunia game. Hal ini dapat disimpulkan bahwa dengan mekanik saat ini, pemain masih merasakan kekurangan pada cerita dari *customer*. Untuk mengidentifikasi lebih dalam permasalahan pada cerita dan narasi pada dialog *customer*, kuisisioner NES (*Narrative Engagement Scale*) disebarkan. Kuisisioner NES sendiri disebarkan dengan tujuan mengidentifikasi dimensi narasi bagian mana yang menjadi kekurangan utama dari game “*Tea’n Brew*”. Berikut hasil kuisisioner NES yang dapat dilihat pada **Tabel 3.6**.

Tabel 3.6 Hasil kuisisioner NES

Dimensi	Presentase	Kategori
<i>Narrative Understanding</i>	69.43%	Tinggi
<i>Attentional Focus</i>	46.10%	Sedang
<i>Emotional Engagement</i>	49.81%	Sedang
<i>Narrative Presence</i>	57.86%	Sedang

Berdasarkan hasil kuisisioner NES, dapat dilihat bahwa dimensi *Narrative Understanding* sudah masuk dalam kategori tinggi yang artinya sebagian besar pemain dapat memahami alur dialog dari game “*Tea’n Brew*” namun, ketiga dimensi lain berada di kategori sedang padahal seharusnya game “*Tea’n Brew*” yang merupakan *game visual novel* memiliki kategori yang tinggi di setiap dimensi NES. Berikut ini kesimpulan ketiga dimensi tersebut:

1. Dimensi *Attentional focus* masuk kategori sedang dengan presentase 46.10%, hal ini menandakan pemain kurang fokus terhadap cerita dan mudah terdistraksi. Kesimpulan dimensi ini ke dalam game yaitu cerita yang diberikan game tidak membuat para pemain merasa cerita tersebut penting.

2. Dimensi *Emotional engagement* masuk kategori sedang dengan presentase 49.81%. Hal ini menandakan bahwa cerita pada *game* masih kurang kuat dari sisi emosional yang membuat pemain kurang merasakan empati terhadap *customer* pada *game* tersebut. Kesimpulannya adalah dialog yang ada di *game* “*Tea 'n Brew*” belum terasa manusiawi dan terkesan kaku sehingga pemain tidak dapat berempati terhadap cerita dari *customer*.
3. Dimensi *Narrative presence* masuk kategori sedang dengan presentase 57.86%. Hal ini menandakan bahwa pemain belum merasa seakan akan berada di dalam dunia *game*. Kesimpulannya adalah *customer* dalam *game* masih belum terasa manusiawi karena tidak memiliki kepribadian yang membuat pemain tidak merasa *customer* tersebut seakan-akan hidup, hal ini disebabkan oleh mekanik *customer* yang diacak oleh sistem begitu pula dengan dialognya yang juga diacak.

Berdasarkan penjabaran masalah berdasarkan *rating* dan *review game* setelah rilis serta kuisioner GEQ dan NES, berikut ini adalah tabel pemetaan masalah dengan alur mekanik *randomize customer* yang ada pada skenario *customer came* pada *game* yang dapat dilihat pada **Tabel 3.7**.

Tabel 3.7 Pemetaan mekanik *randomize customer* terhadap masalah

Masalah	Tahapan
<i>Rating dan review</i>	<i>Randomize Dialogue</i>
Dimensi GEQ <i>Immersion</i>	<i>Randomize Dialogue</i>
Dimensi NES <i>Attentional Focus</i>	<i>Randomize Dialogue</i>
Dimensi NES <i>Emotional Engagement</i>	<i>Randomize Dialogue</i>
Dimensi NES <i>Narrative Presence</i>	<i>Randomize Profile</i>

Terdapat 2 tahapan yang bermasalah yaitu alur *randomize dialogue* dan *randomize profile*. Keduanya merupakan tahapan pada mekanik *randomize customer* sehingga kedua tahapan tersebut saling berkaitan. Pada penelitian sebelumnya sudah memiliki solusi untuk masing-masing masalah [6] dimana untuk *randomize profile* penelitian sebelumnya menggunakan LLM untuk membuat *profile* NPC baru yang memiliki model kepribadian OCEAN, lalu untuk masalah *randomize dialogue* penelitian sebelumnya menggunakan *single agent* LLM untuk menghasilkan dialog yang dinamis. Meskipun masing-masing masalah tersebut sudah memiliki solusi, masih ada kekurangan yaitu pada solusi membuat *profile*

NPC baru pada penelitian sebelumnya masih dibuat saat masa development yang mana *profile* NPC dibuat bukan selama *game* berjalan, namun selama mengembangkan *game*. Sedangkan pada kasus *game* “Tea 'n Brew” baik membuat *profile* NPC dan dialog harus dilakukan secara berurutan selama *game* berjalan. Maka dari itu sistem *multi agent* menjadi solusi yang tepat pada kasus ini untuk mengatasi kedua masalah tersebut.

3.1.2 Persiapan Data

Berdasarkan permasalahan diatas, disiapkan data yang dibutuhkan dalam penelitian ini. Persiapan data dilakukan untuk tahapan merancang *agent* untuk sistem *multi agent*. Berikut merupakan tabel data dapat dilihat pada **Tabel 3.8**.

Tabel 3.8 Data yang dibutuhkan

Data	Tipe Data
Customer	Class
Resep	Class

1. Data Customer

Data *customer* berbentuk class yang berisi 11 variabel yang disimpan di sebuah class bernama *CustomerProfile*. Data ini digunakan sebagai list *profile customer* berdasarkan rentang usia (remaja, dewasa, orang tua) beserta gendernya yang berisi informasi seperti nama, list dialog, dan list aset *sprite*. Masing-masing variabel tersebut akan diacak oleh sistem *game* untuk memunculkan 1 *customer* dengan kombinasi tertentu. Berikut ini merupakan struktur data *customer* saat ini:

CustomerProfile
+ categoryName: string
+ possibleName: List<string>
+ headSprites: List<sprite>
+ hairSprites: List<sprite>
+ shirtSprites: List<sprite>
+ orderOption: List<CustomerOrderOption>
+ curhatStories: List<TextAsset>
+ minMaxFails: int
+ maxMaxFails: int
+ minStartingMood: int
+ maxStartingMood: int

Berikut ini adalah contoh *instance customer* dengan kategori *Teen Male*:

Variable	Value
categoryName	“Teen Male”

Variable	Value
possibleName	["Kevin", "Clyde", "Clark", "Paul", "Law", "Chris", "Finn", "Mich", "Carl", "Hal", "Barry", "Bruce", "Tony", "Steve", "Arthur"]
headSprites	
hairSprites	
shirtSprites	
orderOption	{["Green Tea", Green Tea.json], ["Black Tea", Black Tea.json], ["Mint Tea", Mint Tea.json], ["Apple Tea", Apple Tea.json], ["Lavender Tea", Lavender Tea.json], ["Milk Tea", Milk Tea.json], ["Honey Tea", Honey Tea.json], ["Jasmine Tea", Jasmine Tea.json]}
curhatStories	[Curhat 1.json, Curhat 2.json, Curhat 3.json, Curhat 4.json, Curhat 5.json]
minMaxFails	2
maxMaxFails	4
minStartingMood	30
maxStartingMood	50

2. Data Resep

Data resep berbentuk class yang terdiri dari 4 variabel yang disimpan pada sebuah class bernama *teaRecipe*. Data ini digunakan untuk menyimpan detail resep seperti nama, bahan yang digunakan, gambar hasil minuman, dan deskripsi minuman. Data resep ini akan digunakan oleh sistem *game* sebagai acuan pemain untuk membuat minuman.

TeaRecipe
+ recipeName: string
+ ingredients: List<string>
+ resultSprite: sprite
+ recipeDescription: string

Berikut ini adalah contoh *instance* resep minuman *Black Tea*:

Variable	Value
recipeName	"Black Tea"
ingredients	["Black Tea Leaf", "Hot Water"]
resultSprite	
recipeDescription	Seduhan klasik dengan aroma earthy yang kuat dan sentuhan rasa pahit yang bersih. Karakter teh yang pekat dan hangat ini memberikan sensasi ketenangan instan, sangat pas untuk menjernihkan pikiran yang sedang semrawut.

3.1.3 Sistem Multi Agent

Pada tahapan ini, dilakukan analisis untuk membangun sistem *multi agent* secara menyeluruh mulai dari menentukan *agent* apa saja yang akan dibuat, menentukan tugas *agent*, dan menentukan alur sistem *multi agent* menggunakan *framework* tertentu.

3.1.3.1 Task Decomposition

Berdasarkan masalah yang sudah dijabarkan pada tahapan domain understanding, terdapat 2 masalah pada tahapan dari mekanik *randomize customer* yaitu *randomize dialogue* dan *randomize profile*. Dari kedua masalah tersebut maka akan dirancang *agent* apa saja yang cocok untuk menyelesaikan masing-masing masalah. Berikut adalah tabel pemetaan masalah ke *agent* yang dapat dilihat pada Tabel 3.9.

Tabel 3.9 Pemetaan masalah ke *agent*

Masalah	Agent
<i>Randomize profile</i>	<i>Profile Agent</i>
<i>Randomize dialogue</i>	<i>Dialogue Agent & Critic Agent</i>

Khusus untuk *randomize dialogue* akan dibuat 2 *agent* menggunakan pola actor critic dikarenakan dalam proses pembuatan dialog memerlukan adanya proses validasi bahwa dialog yang dibuat sudah sesuai dengan *personality customer* dari yang sebelumnya dilakukan oleh tim pengembang *game* sekarang dilakukan oleh *agent*. Dengan pembagian tersebut maka ada 3 *agent* yang akan dibuat untuk menyelesaikan masalah yaitu *profile agent*, *dialogue agent*, dan *critic agent*.

3.1.3.2 Analisis Agent

Pada tahapan ini dilakukan analisis setiap *agent* yang sudah ditentukan pada tahapan *task decomposition*. Berikut ini adalah proses analisis setiap *agent*:

3.1.3.2.1 Profile Agent

Tugas utama dari *Profile Agent* adalah menghasilkan *profile* dari *customer* yang baru saja datang ke *cafe* "Tea'n Brew" secara dinamis. Karena *agent* ini beroperasi di latar belakang sebelum *customer* muncul di layar, tugas spesifiknya meliputi membuat *profile customer* mulai dari nama, latar belakang *customernya*, masalah yang dihadapi *customer*, serta *score* kepribadian OCEAN dalam skala 0 sampai 100.

Penentuan Model LLM yang Digunakan

Berdasarkan kebutuhannya untuk membuat *profile customer* dibutuhkan LLM yang dapat mengikuti struktur yang diberikan, mampu menghasilkan output dengan bahasa indonesia yang baik, serta mampu men-*generate* output dengan cepat. Berikut tabel perbandingan performansi model yang dapat dilihat pada **Tabel 3.10**.

Tabel 3.10 Peforma model untuk *profile agent*

Aspek	GPT-5.4 nano	GPT-5.4 mini	GPT-5.5
Intelligence	38	40	50
Biaya/NPC	\$0.006	\$0.01	\$0.01
Kecepatan	166 t/s	183 t/s	74 t/s
Latency	2.4 detik	2.2 detik	5.4 detik
JSON Valid	perlu test	bagus	paling konsisten
Bahasa Indonesia	cukup	bagus	terbaik

GPT 5.4 Mini dipilih karena secara peforma dapat memenuhi semua kebutuhan yang diperlukan *agent*. Secara peforma GPT 5.4 Mini masih mampu menyaingi kemampuan GPT 5.5 yang dapat dilihat lebih tinggi dari beberapa aspek, namun GPT 5.4 Mini memiliki keunggulan pada kecepatan dan latensi yang membuat output model ini menjadi jauh lebih cepat [17].

System Prompt

System prompt ini berfungsi untuk memberi *profile agent* tugas dan instruksi apa saja yang harus dilakukan serta aturan yang harus ditaati *agent*. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.11**.

Tabel 3.11 Pemetaan *system prompt profile agent*

Komponen	Detail
<i>Introduction</i>	Memberikan peran kepada <i>agent</i> dimana dia berperan sebagai <i>profile agent</i> dan menjelaskan tugas <i>agent</i> untuk membuat <i>profile customer</i> setiap kali dipanggil.
<i>Individual Prompt Elements</i>	Memberikan elemen <i>prompt</i> berupa aturan-aturan terkait <i>profile customer</i> yang akan dibuat yaitu, nama <i>customer</i> hanya perlu dibuat 1 kata saja, menjelaskan latar belakang <i>customer</i> , membuat masalah yang dihadapi dengan lebih spesifik, menjabarkan kepribadian <i>customer</i> berdasarkan <i>score</i> OCEAN dari skala nilai 0-100, dan gaya bahasa sesuai dengan kepribadian <i>customer</i> .
<i>Refocus</i>	Memberikan instruksi ulang bahwa <i>agent</i> harus membuat <i>profile customer</i> berdasarkan aturan-aturan yang sudah dijelaskan sebelumnya tanpa terkecuali.

Komponen	Detail
Transition	Memberikan perintah transisi dimana agent harus membuat <i>profile customer</i> berdasarkan format JSON yang valid, bukan format lainnya seperti <i>markdown</i> .

Berdasarkan hasil pemetaan diatas dihasilkan *system prompt* untuk *profile agent* sebagai berikut:

Introduction [Game Fiction Context — Tea'n Brew Visual Novel] Tugas: merakit profil satu <i>customer</i> café yang unik, realistis, dan relatable. # Individual Prompt Elements ATURAN WAJIB: - Nama: HANYA satu kata (nama panggilan). TANPA nama belakang. Contoh OK: Raka, Dimas, Sari, Bunga, Lesti, Fajar, Tari, Joko. Contoh SALAH: Raka Pratama, Dimas Putra. - Background: tepat 2 kalimat. Sertakan: - Apa yang dilakukan NPC sehari-hari (pekerjaan/status) - Satu fakta unik tentang kepribadiannya - Masalah hari ini: HARUS spesifik dan berbeda antar NPC. JANGAN pernah pakai masalah generik. Pilih SATU tema dari daftar ini: - Tugas kuliah/kantor yang deadline-nya besok - Pertengkaran dengan sahabat yang belum diselesaikan - Bingung harus memilih A atau B dalam hidup - Dimarahi orang tua tadi pagi tentang sesuatu - Dompot kosong tapi harus bayar sesuatu minggu ini - Putus cinta atau bertengkar dengan pacar - capek banget tapi masih harus hadir ke acara - Diejek atau diremehkan orang hari ini - Pindah ke tempat baru dan belum punya teman - Khawatir gagal ujian/tes/proyek penting Setelah memilih tema, JELASKAN secara spesifik — misalnya bukan "masalah kuliah" tapi "deadline skripsi yang dimajukan dosen padahal baru setengah jadi". - OCEAN (0-100): buat <i>score</i> yang KONSISTEN. Contoh: - Introvert (E<40) + Neurotic tinggi → masalah sosial terasa lebih berat - Extrovert (E>60) + Agreeable tinggi → masalahnya tentang orang lain, bukan diri sendiri - Openness tinggi → masalahnya kreatif/unik, bukan rutinitas - max_fails (1-3): dipengaruhi Agreeableness + Neuroticism: - A tinggi + N rendah → max_fails=3 (sabar) - A rendah + N tinggi → max_fails=1 (cepat marah) - Sisanya → max_fails=2 - reaksi_gaya: pilih SATU cara NPC bereaksi saat marah dan saat puas.
--

Refocus

Berdasarkan aturan-aturan tersebut. Buatlah *profile customer* tanpa mengecualikan satupun aturan tersebut.

Transition

Kembalikan HANYA JSON valid, tanpa markdown, tanpa teks lain.

User Prompt

User prompt ini akan menggunakan data *customer* yang berasal dari *class* *CustomerProfile* sebagai input data. Tujuan dari *prompt* ini adalah untuk menghasilkan *profile customer* dalam format JSON yang nantinya akan digunakan untuk membuat dialog. Berikut ini adalah pemetaan komponen *prompt* dan detail komponen *prompt* yang dapat dilihat pada **Tabel 3.12**.

Tabel 3.12 Pemetaan user prompt profile agent

Komponen	Detail
<i>Introduction</i>	Memberikan <i>agent</i> tugas untuk membuat <i>profile customer</i> saat <i>agent</i> dipanggil dengan mengambil data <i>customer</i> dari <i>class</i> <i>CustomerProfile</i> yaitu {categoryName}.
<i>Individual Prompt Elements</i>	Memberikan format yang harus ditaati <i>agent</i> beserta dengan contoh outputnya.
<i>Refocus</i>	Memberikan contoh <i>profile customer</i> lainnya dengan format yang sama dengan sebelumnya untuk memastikan model tidak mengarang format yang diberikan
<i>Transition</i>	Menegaskan kembali format JSON beserta aturan-aturan yang harus dipenuhi pada setiap elemennya.

Berdasarkan hasil pemetaan diatas dihasilkan *user prompt* untuk *profile agent* sebagai berikut:

Introduction

Customer baru memasuki *cafe*. Buat profilnya:
- Kategori usia : {categoryName}

Individual Prompt Elements

Contoh profil BAIK:

```
{
  "nama": "Raka",
  "background": "Mahasiswa teknik yang hobi main gitar di kosan. Tipikal orang yang keliatan cuek tapi sebenarnya perhatian banget.",
  "masalah_hari_ini": "Deadline tugas kelompok dimajukan jadi besok padahal anggota timnya pada menghilang semua, bingung harus mulai dari mana.",
  "ocean": { "openness": 62, "conscientiousness": 55, "extraversion": 68, "agreeableness": 47, "neuroticism": 61 },
  "max_fails": 2,
```

```

"reaksi_gaya": "langsung ngomong keras dan minta bicara sama manajernya"
}
# Refocus
Contoh profil LAINNYA (jangan duplikat):
{
  "nama": "Sari",
  "background": "Barista magang yang baru kerja seminggu. Masih suka nervous kalau ada customer marah.",
  "masalah_hari_ini": "Ketahuan bosnya suka main HP pas kerja dan sekarang dipanggil masuk ruangan buat dimarahin.",
  "ocean": { "openness": 45, "conscientiousness": 38, "extraversion": 52, "agreeableness": 72, "neuroticism": 75 },
  "max_fails": 1,
  "reaksi_gaya": "muka merah padam, bisik bilang 'ya udahlah' terus langsung pergi"
}

# Transition
Format JSON:
{
  "nama": "satu kata saja",
  "background": "kalimat 1. kalimat 2.",
  "masalah_hari_ini": "masalah SPESIFIK yang dibawa hari ini (bukan generik)",
  "ocean": {
    "openness": <0-100>,
    "conscientiousness": <0-100>,
    "extraversion": <0-100>,
    "agreeableness": <0-100>,
    "neuroticism": <0-100>
  },
  "max_fails": <1, 2, atau 3>,
  "reaksi_gaya": "1 cara NPC bereaksi saat marah"
}

```

3.1.3.2.2 Dialogue Agent

Tugas *Dialogue Agent* sebagai "Actor" dalam pola *Actor-Critic* adalah menghasilkan narasi teks yang dinamis dan manusiawi berdasarkan *profile customer* hasil *generate* dari *Profile Agent*. *Agent* ini harus memastikan bahwa setiap kata yang diucapkan NPC mencerminkan *score* kepribadian OCEAN, latar belakang, dan masalah spesifik yang dibawa oleh *customer* tersebut. Dialog pada game "*Tea 'n Brew*" sendiri terbagi menjadi beberapa bagian yang dapat dilihat pada **Tabel 3.13:**

Tabel 3.13 Jenis dialog pada game “Tea’n Brew”

Jenis Dialog	Penjelasan	Skenario
<i>Order Dialogue</i>	Merupakan dialog yang muncul pertama kali saat NPC datang	<i>Customer Came</i>
<i>False Order Dialogue</i>	Merupakan dialog yang muncul bila pemain salah memberi pesanan namun jumlah kesalahan belum mencapai max_fails.	<i>Make Tea</i>
<i>Failed Order Dialogue</i>	Merupakan dialog bila pemain salah memberi pesanan sebanyak variabel max_fails.	<i>Make Tea</i>
<i>Success Order Dialogue</i>	Merupakan dialog yang muncul bila pemain membuat pesanan dengan tepat.	<i>Make Tea</i>
<i>Venting Dialogue</i>	Merupakan dialog yang muncul pada sesi curhat yang sudah termasuk pilihan dialog yang dipilih pemain.	<i>Venting Session</i>
<i>Venting Reaction Dialogue</i>	Merupakan dialog reaksi yang muncul berdasarkan pilihan pemain.	<i>Venting Session</i>
<i>Venting Closing Dialogue</i>	Merupakan dialog penutup yang diberikan customer sebelum customer pergi.	<i>Venting Session</i>

Penentuan Model LLM yang Digunakan

Berdasarkan kebutuhannya untuk membuat dialog *customer* dibutuhkan LLM yang juga dapat mengikuti struktur yang diberikan, mampu menghasilkan bahasa indonesia yang natural, kreatif dalam menghasilkan *input* namun tetap konsisten dengan kepribadian NPC, serta mampu mengenerate *output* dengan cepat. Berikut tabel perbandingan performa model yang dapat dilihat pada **Tabel 3.14**.

Tabel 3.14 Performa model untuk *dialogue agent*

Aspek	GPT-5.4 nano	GPT-5.4 mini	GPT-5.5
Intelligence	38	40	50
Biaya/NPC	\$0.006	\$0.01	\$0.01
Kecepatan	166 t/s	183 t/s	74 t/s
Latency	2.4 detik	2.2 detik	5.4 detik
JSON Valid	perlu test	bagus	paling konsisten
Bahasa Indonesia	cukup	bagus	terbaik

Sama seperti *profile agent*, GPT 5.4 Mini dipilih karena secara performa dapat memenuhi semua kebutuhan yang diperlukan *agent*. Performa GPT 5.4 Mini masih mampu menyaingi kemampuan GPT 5.5 yang dapat dilihat lebih tinggi dari beberapa aspek, namun GPT 5.4 Mini memiliki keunggulan pada kecepatan dan latensi yang membuat output model ini menjadi jauh lebih cepat [17].

Commented [sr1]: sebaiknya penentuan model sekaligus diawal, jangan per agent

System Prompt

System prompt ini berfungsi untuk memberi *dialogue agent* tugas dan instruksi apa saja yang harus dilakukan serta aturan yang harus ditaati *agent*. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.15**.

Tabel 3.15 Pemetaan *system prompt dialogue agent*

Komponen	Detail
<i>Introduction</i>	Memberikan peran kepada <i>agent</i> dimana dia berperan sebagai <i>dialogue agent</i> dan menjelaskan tugas <i>agent</i> untuk membuat dialog berdasarkan <i>profile customer</i> yang sudah dibuat oleh <i>profile agent</i>
<i>Individual Prompt Elements</i>	Memberikan elemen <i>prompt</i> berupa penjelasan terkait dialog <i>customer</i> yang akan dibuat yaitu, dialog yang muncul harus sesuai dengan kepribadian OCEAN <i>customer</i> serta gaya bahasa yang sesuai dengan <i>profile customernya</i> .
<i>Refocus</i>	Memberikan instruksi untuk fokus terkait aturan yang harus terpenuhi saat membuat dialog.
<i>Transition</i>	Memberikan perintah dimana <i>agent</i> harus membuat dialog <i>customer</i> berdasarkan format JSON yang valid.

Berdasarkan hasil pemetaan diatas dihasilkan *system prompt* untuk *dialogue agent* sebagai berikut:

Introduction [Game Fiction Context - Tea'n Brew Visual Novel] Tugas: menghasilkan dialog NPC yang AUTENTIK dan TERASA HIDUP berdasarkan kepribadian OCEAN-nya. # Individual Prompt Elements KEPRIBADIAN NPC (DITERJEMAHKAN ke perilaku bicara): {build_ocean_desc(ocean)} POLA BICARA NPC BERDASARKAN OCEAN: - Kekuatan/Ekspresi : {ekspresi_e} - Emosional : {ekspresi_n} - Kelembutan/Konflik : {ekspresi_a} - Kreativitas bahasa: {ekspresi_o} Gaya bahasa usia ({usia}): {GAYA_BAHASA.get(usia, 'natural')} # Refocus ATURAN MUTLAK: 1. NPC memanggil pemain "kak". JANGAN "bro", "man", "sis", "guys". 2. Dialog harus TERASA BEDA antar NPC. Jangan semua NPC bicara dengan nada yang sama. 3. Tunjukkan OCEAN melalui CARA BICARA, bukan dengan menyebutkan angka atau sifat langsung.

4. Kalimat harus natural seperti orang bicara sungguhan — ada jeda, ada emosi, ada gaya.
5. Untuk state curhat: NPC wajib merespons jawaban pemain di ronde sebelumnya, jangan mulai dari awal.

Transition

Kembalikan HANYA JSON valid, tanpa teks lain.

User Prompt

Berdasarkan kasus game “*Tea’n Brew*” yang memiliki 7 jenis dialog yang berbeda, maka dibuat pula 6 user *prompt* berbeda yang masing-masing digunakan sebagai input ke *dialogue agent* untuk menghasilkan 7 dialog sesuai jenisnya. Berikut ini adalah user *prompt* untuk masing-masing jenis dialog:

1. *User Prompt Order Dialogue*

User *prompt* ini berfungsi untuk memberi *dialogue agent* tugas dan instruksi pada skenario *customer came* dimana *customer* datang dan memesan minuman. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.16**.

Tabel 3.16 Pemetaan user prompt order dialogue

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile agent</i> beserta minuman yang dipesan yang berasal dari <i>class teaRecipe</i> untuk membuat dialog awal saat <i>customer</i> memesan minuman.
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan <i>agent</i> agar membuat dialog sesuai dengan yang dijelaskan
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar <i>agent</i> membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah <i>agent</i> agar menghasilkan <i>output</i> sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan User *prompt* untuk *dialogue agent* sebagai berikut:

Introduction NPC: {state['nama']} ({state['usia']}, {state['gender']}) Background : {state['background']} Masalah hari ini: {state['masalah_hari_ini']} Menu tersedia: {'', 'join(MENU_MINUMAN)} # Individual Prompt Elements Buat dialog NPC PERTAMA KALI datang ke café dan memesan minuman. PANDUAN: - NPC sedang membawa beban masalah hari ini, jadi dialog pesannya boleh terpengaruh suasana hati - Misalnya: kalau lagi sedih mungkin pesannya sambil mendesah, kalau lagi marah mungkin pesannya to the point - Tapi JANGAN langsung cerita masalah — cukup terlihat dari nada atau ekspresi kecil - Pilih minuman dari menu yang sesuai suasana hati NPC - 1-3 kalimat, natural, sesuai pola bicara OCEAN di atas - Panggil pemain "kak" (1 kali saja) # Refocus Format JSON: <pre>{ "minuman_dipesan": "nama minuman dari menu", "dialog_pesanan": "dialog NPC memesan..." }</pre> # Transition Buat dalam format JSON yang valid.

2. User Prompt False Order Dialogue

User *prompt* ini berfungsi untuk memberi *dialogue agent* tugas dan instruksi untuk membuat dialog bila pesanan salah pada skenario *make tea*. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.17**.

Tabel 3.17 Pemetaan user prompt False Order Dialogue

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile agent</i> beserta minuman yang dipesan yang berasal dari <i>class teaRecipe</i> untuk membuat dialog bila pemain salah memberi minuman yang dipesan <i>customer</i> .
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan <i>agent</i> agar membuat dialog sesuai dengan yang dijelaskan
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar <i>agent</i> membuat sesuai format tersebut

Komponen	Detail
Transition	Memberikan perintah agent agar menghasilkan <i>output</i> sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan *User prompt* untuk *dialogue agent* sebagai berikut:

Introduction NPC: {state['nama']} ({state['usia']}, {state['gender']}) Pesanan NPC: {state.get('minuman_dipesan', 'teh')} Kesalahan ke-{fail_count} dari maksimal {max_fails} kali. Intensitas: {intensitas}{gaya_hint} # Individual Prompt Elements Pemain menyajikan minuman yang SALAH. CARA NPC BEREAKSI BERDASARKAN KEPALIBADIAN: - Kalau Agreeableness tinggi: kecewanya lembut, lebih ke sedih dari marah. "ih kak, ini bukan yang aku pesen..." - Kalau Agreeableness rendah: frontal dan blak-blakan. "eh kok salah? yang bener dong!" - Kalau Neuroticism tinggi: reaksi berlebihan, dramatis. "parah banget sih kak, udah kedua kalinya!" - Kalau Neuroticism rendah: tetap tenang meski kesal. "hmm ini beda ya kak, coba lagi?" - Kalau Extraversion tinggi: bere keras, langsung komplain ke siapa saja. - Kalau Extraversion rendah: diam, cemberut, tidak manya manya tapi terlihat kecewa. # Refocus Sesuaikan dengan intensitas di atas. Maks 2 kalimat. Format JSON: <pre>{ "dialog_pesanan_salah": "reaksi NPC saat pesanan salah..." }</pre> # Transition Buat dalam format JSON yang valid.

3. *User Prompt Failed Dialogue*

User prompt ini berfungsi untuk memberi *dialogue agent* tugas dan instruksi untuk membuat dialog bila pemain gagal membuatkan pesanan yang tepat pada skenario make tea. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.18**.

Tabel 3.18 Pemetaan *user prompt Failed Order Dialogue*

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile</i> agent untuk membuat dialog pesanan gagal saat pemain memberikan pesanan yang salah sebanyak variabel {max_fails}
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent agar membuat dialog sesuai dengan yang dijelaskan
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar agent membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan *User prompt* untuk *dialogue agent* sebagai berikut:

Introduction NPC: {state['nama']} ({state['usia']}, {state['gender']}) Pesanan NPC: {state.get('minuman_dipesan', 'teh')} NPC sudah salah {state.get('max_fails', 2)} kali dan kehabisan sabar. Cara marah: {cara_marah}{gaya_hint} # Individual Prompt Elements Pemain gagal melayani pesanan NPC untuk yang terakhir kalinya. Buat dialog NPC MARAH dan memutuskan PERGI dari café. Sesuaikan cara marah dengan OCEAN. Maksimal 2-3 kalimat. # Refocus Format JSON: { "dialog_marah": "dialog NPC marah dan pergi..." } # Transition Buat dalam format JSON yang valid.

4. User Prompt Success Dialogue

User *prompt* ini berfungsi untuk memberi *dialogue agent* tugas dan instruksi untuk membuat dialog bila pemain berhasil membuatkan pesanan yang tepat pada skenario make tea. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.19**.

Tabel 3.19 Pemetaan *user prompt Success Order Dialogue*

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile</i> agent untuk membuat dialog bila pemain berhasil membuat pesanan yang tepat.
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent agar membuat dialog sesuai dengan yang dijelaskan

Komponen	Detail
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar agent membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan *User prompt* untuk *dialogue agent* sebagai berikut:

Introduction NPC: {state['nama']} ({state['usia']}, {state['gender']}) Pesanan NPC: {state.get('minuman_dipesan', 'teh')} Pemain menyajikan minuman BENAR. Gaya senang: {gaya_senang}{reaksi_plus} # Individual Prompt Elements Buat dialog NPC puas menerima pesannya. Tunjukkan kepuasan melalui CARA BICARA NPC sesuai OCEAN. 1-3 kalimat. Panggil pemain "kak" (1 kali). JANGAN lanjut ke topik lain. Berhenti setelah ekspresi kepuasan. # Refocus Format JSON: { "dialog_berhasil": "dialog NPC puas dengan pesanan..." } # Transition Buat dalam format JSON yang valid.
--

5. *User Prompt Venting Dialogue*

User *prompt* ini berfungsi untuk memberi *dialogue agent* tugas dan instruksi untuk membuat dialog setelah *success dialogue* dimana *customer* akan mulai curhat pada pemain sesuai pada skenario *venting session*. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.20**.

Tabel 3.20 Pemetaan user prompt Venting Dialogue

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile</i> agent untuk membuat dialog curhatan <i>customer</i> beserta 3 pilihan yang dapat pemain pilih
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent agar membuat dialog sesuai dengan yang dijelaskan beserta 3 pilihan untuk pemain pilih yang akan menghasilkan reaksi berbeda dari <i>customer</i>

Komponen	Detail
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar agent membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan User *prompt* untuk *dialogue agent* sebagai berikut:

Introduction NPC: {state['nama']} ({state['usia']}, {state['gender']}) Masalah utama: {state['masalah_hari_ini']} Mood NPC: {mood}/100 - {_mood_ctx(mood)} Ronde: {ronde} dari {total_ronde} {_riwayat_ctx(riwayat)} {konteks_sebelumnya} # Individual Prompt Elements Buat dialog curhat NPC untuk ronde ini. PANDUAN PENTING: - Tulis seperti orang sungguhan sedang curhat, bukan ringkasan cerita - {"RONDE 1: Perkenalkan masalah SECARA NATURAL. Sebutkan SIAPA yang terlibat (sahabat, teman, pacar, keluarga) dan APA yang terjadi. Tunjukkan EMOSI NPC melalui cara bercerita." if ronde == 1 else "LANJUTKAN cerita. Merespons jawaban pemain, lalu dalami cerita lebih jauh. Tunjukkan perubahan emosi berdasarkan jawaban pemain.\n\nATURAN CHRONOLOGICAL: Cerita HARUS BERGERAK MAJU dalam waktu. Setiap ronde adalah KEJADIAN BARU yang terjadi SETELAH ronde sebelumnya. JANGAN flashback ke masa lalu atau mengulang kejadian yang sudah diceritakan. Fokus ke dampak/consequence dari masalah, bukan mengulang kronologi yang sama."} - Variasi angle: {tema_pilihan} - JANGAN mengulang data/fakta/plot point yang sudah disebut di riwayat (lihat list DATA YANG SUDAH DISEBUT di atas) - JANGAN mengakhiri cerita jika bukan ronde terakhir - Minimal 3-5 kalimat yang mengalir natural INFLUENSI MOOD TERHADAP CARA BERCERITA: - Mood >=70: Cerita dengan antusias, ada unsur positif, banyak detail, suka pakai "banget", "parah deh", nada ceria meski ceritanya sedih - Mood 40-69: Cerita apa adanya, proporsional, tidak berlebihan - Mood 20-39: Cerita pendek, kalimat putus-putus, sering diam atau menunda, nada pesimis - Mood <20: Sangat pendek, kadang hanya 1-2 kalimat, tidak mau banyak cerita, nada menyerah - Panggil pemain "kak" secara natural (1-2 kali saja, jangan berlebihan) PANDUAN pilihan_jawaban (WAJIB sesuai mood NPC): - Saat mood tinggi (>=60): pilihan satisfy harus lebih hangat, neutral netral, angry bisa lebih ringan - Saat mood rendah (<40): pilihan satisfy harus lebih sabar/empati, angry bisa lebih tajam - satisfy: empati tulus, paham perasaan NPC, validasi emosinya. BUKAN saran solusi. - neutral: respons biasa, memberi saran umum, tidak terlalu dalam. BUKAN acuh tak acuh total.

- angry: menyepelkan, menyuruh ribet, atau respons yang kurang peka. TAPI tetap realistis.
- JANGAN gunakan "bro" atau sebutan akrab lainnya di pilihan jawaban
- Pilihan harus MERESPONS langsung apa yang NPC baru saja ceritakan

Refocus

Format JSON:

```
{
  "dialog_npc": "cerita NPC yang mengalir...",
  "pilihan_jawaban": [
    {"id": 1, "teks": "...", "nada": "satisfy"},
    {"id": 2, "teks": "...", "nada": "neutral"},
    {"id": 3, "teks": "...", "nada": "angry"}
  ]
}
```

Transition

Buat dalam format JSON yang valid.

6. User Prompt Venting Reaction Dialogue

User *prompt* ini berfungsi untuk memberi dialogue agent tugas dan instruksi untuk membuat dialog reaksi dari *customer* setelah pemain memilih 1 dari 3 pilihan jawaban yang ada pada skenario venting session. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.21**.

Tabel 3.21 Pemetaan user prompt Venting Reaction Dialogue

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile</i> agent serta dialog yang dipilih pemain untuk menghasilkan dialog reaksi yang sesuai.
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent agar membuat dialog reaksi sesuai pilihan pemain dan karakter dari <i>customer</i> .
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar agent membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan User *prompt* untuk *dialogue agent* sebagai berikut:

Introduction

```
NPC: {state['nama']} ({state['usia']}, {state['gender']})
Masalah utama: {state['masalah_hari_ini']}
Mood NPC: {mood}/100 - {_mood_ctx(mood)}
RONDE TERAKHIR ({ronde}/{total_ronde})
{_riwayat_ctx(riwayat)}
{konteks_sebelumnya}
```

Individual Prompt Elements Buat PENUTUP cerita NPC. Ini adalah ronde terakhir. PANDUAN: - Merespons jawaban pemain dari ronde sebelumnya - Berikan resolusi, pelajaran, atau penutup emosional dari cerita - Mood tinggi (≥ 60): penutup hangat, penuh harapan, terima kasih tulus - Mood rendah (< 40): penutup datar, pasrah, atau bahkan agak sinis - Tutup dengan natural, boleh terima kasih atau harapan kecil - 3-4 kalimat yang mengalir natural - Panggil pemain "kak" secara natural (1-2 kali saja) # Refocus Format JSON: <pre>{ "dialog_npc": "penutup cerita NPC yang natural...", "pilihan_jawaban": [] }</pre> # Transition Buat dalam format JSON yang valid.

7. User Prompt Venting Closing Dialogue

User *prompt* ini berfungsi untuk memberi dialogue agent tugas dan instruksi untuk membuat dialog penutup bisa ronde curhat sudah mencapai akhir yang ada pada skenario venting session. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.22**.

Tabel 3.22 Pemetaan user prompt Venting Closing Dialogue

Komponen	Detail
<i>Introduction</i>	Memberikan data yang akan digunakan sebagai <i>profile customer</i> hasil dari <i>profile</i> agent serta dialog yang dipilih pemain untuk menghasilkan dialog penutup yang sesuai.
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent agar membuat dialog sesuai karakter dari <i>customer</i> .
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar agent membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan User *prompt* untuk *dialogue agent* sebagai berikut:

Introduction NPC: {state['nama']} ({state['usia']}, {state['gender']}) Masalah utama: {state['masalah_hari_ini']} Mood NPC: {mood}/100 - {_mood_ctx(mood)}
--

```

RONDE TERAKHIR ({ronde}/{total_ronde})
{_riwayat_ctx(riwayat)}
{konteks_sebelumnya}

# Individual Prompt Elements
Buat PENUTUP cerita NPC. Ini adalah ronde terakhir.

PANDUAN:
- Merespons jawaban pemain dari ronde sebelumnya
- Berikan resolusi, pelajaran, atau penutup emosional dari cerita
- Mood tinggi (>=60): penutup hangat, penuh harapan, terima kasih tulus
- Mood rendah (<40): penutup datar, pasrah, atau bahkan agak sinis
- Tutup dengan natural, boleh terima kasih atau harapan kecil
- 3-4 kalimat yang mengalir natural
- Panggil pemain "kak" secara natural (1-2 kali saja)

# Refocus
Format JSON:
{
  "dialog_npc": "penutup cerita NPC yang natural...",
  "pilihan_jawaban": []
}

# Transition
Buat dalam format JSON yang valid.

```

3.1.3.2.3 Critic Agent

Critic Agent berfungsi sebagai "Quality Gate" (gerbang kualitas) dalam arsitektur *Actor-Critic* yang diterapkan pada sistem ini. Tugas utama agen ini adalah melakukan evaluasi kritis terhadap draf dialog yang dihasilkan oleh *Dialogue Agent* (Actor) sebelum dialog tersebut ditampilkan kepada pemain. Agent ini bertindak sebagai evaluator yang memastikan adanya konsistensi naratif antara isi dialog dengan profile *customer* (*score* OCEAN, latar belakang, dan masalah hari ini) yang telah dibuat oleh *Profile Agent*.

Penentuan Model LLM yang Digunakan

Berdasarkan kebutuhannya untuk mengevaluasi dialog *customer* dibutuhkan LLM yang memiliki intelektual terbaik dan mampu memahami bahasa Indonesia dengan baik. Berikut tabel perbandingan performa model yang dapat dilihat pada **Tabel 3.23**.

Tabel 3.23 Peforma model untuk *critic agent*

Aspek	GPT-5.4 nano	GPT-5.4 mini	GPT-5.5
Intelligence	38	40	50
Biaya/NPC	\$0.006	\$0.01	\$0.01
Bahasa Indonesia	cukup	bagus	terbaik

Model yang dipilih adalah GPT 5.5 karena secara peforma yang dibutuhkan model ini merupakan model terbaik dibandingkan model lain yang dibandingkan. Model ini memiliki peforma intelektual terbaik dan mampu memahami bahasa indonesia dengan lebih baik dibandingkan model lain [17].

System Prompt

System *prompt* ini berfungsi untuk memberi agent tugas untuk menilai dan proses serta kriteria dalam menilai dialog yang akan dihasilkan oleh dialogue agent. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada Tabel 3.24.

Tabel 3.24 Pemetaan system *prompt critic agent*

Komponen	Detail
<i>Introduction</i>	Memberikan peran dan tugas pada agent untuk memvalidasi dialog beserta <i>profile customer</i> dan <i>score ocean</i>
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent dalam melakukan penilaian dengan memberikan kriteria penilaian
<i>Refocus</i>	Memberikan penegasan bahwa <i>score</i> yang dibuat harus sesuai fakta dialognya, maka dari itu <i>score</i> rendah akan tetap diterima
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan *system prompt* untuk *critic agent* sebagai berikut:

Introduction [Game Fiction Context — Tea'n Brew Visual Novel] Tugas: memvalidasi dialog NPC terhadap profil OCEAN, gaya bahasa usia/gender, dan emosi yang seharusnya muncul pada dialog state tersebut. # Individual Prompt Elements KRITERIA PENILAIAN: 1. OCEAN konsisten (40 poin) — Apakah cara bicara NPC sesuai <i>score</i> OCEAN-nya? 2. Emosi tepat (30 poin) — Apakah emosi yang muncul sesuai state dialog? 3. Gaya bahasa usia (20 poin) — Apakah bahasa sesuai kategori usia? 4. Konsistensi karakter (10 poin) — Apakah NPC terasa konsisten, tidak berubah-ubah? # Refocus <i>SCORE KETAT</i>: Jangan ragu memberi <i>score</i> RENDAH jika dialog terasa generik atau tidak menunjukkan personality yang jelas. Dialog yang "aman tapi tidak berkesan" seharusnya <i>score</i>nya 60-75, bukan 85+.
--

Transition

Kembalikan HANYA JSON valid, tanpa teks lain.

User Prompt

User prompt ini berfungsi untuk memberi critic agent tugas dan instruksi untuk menilai dialog yang sudah dibuat oleh dialogue agent. Berikut ini merupakan pemetaan komponen *prompt* dan detail masing-masing komponen yang dapat dilihat pada **Tabel 3.25**.

Tabel 3.25 Pemetaan user prompt Venting Reaction Dialogue

Komponen	Detail
<i>Introduction</i>	Memberikan <i>profile customer</i> hasil serta dialog yang dihasilkan oleh dialogue agent untuk persiapan penilaian
<i>Individual Prompt Elements</i>	Memberikan arahan dan panduan agent agar menilai berdasarkan 4 kriteria penilaian serta memberikan contoh <i>score</i>
<i>Refocus</i>	Memberikan format JSON sebagai arahan agar agent membuat sesuai format tersebut
<i>Transition</i>	Memberikan perintah agent agar menghasilkan output sesuai format

Berdasarkan hasil pemetaan diatas dihasilkan User *prompt* untuk *dialogue agent* sebagai berikut:

Introduction

Validasi dialog NPC (state: {konteks}):

=== PROFIL NPC ===

Nama : {state.get('nama', '')} | Usia: {state.get('usia', '')} | Gender: {state.get('gender', '')}

OCEAN :

Openness : {ocean['openness']}/100

Conscientiousness: {ocean['conscientiousness']}/100

Extraversion : {ocean['extraversion']}/100

Agreeableness : {ocean['agreeableness']}/100

Neuroticism : {ocean['neuroticism']}/100

Gaya bahasa: {GAYA_BAHASA.get(state.get('usia', ''), 'natural')}

Emosi yang diharapkan: {EMOSI_EXPECTED.get(konteks, 'sesuai konteks')}

Individual Prompt Elements

PERILAKU YANG HARUS TERLIHAT BERDASARKAN OCEAN:

{behavior_str}

=== DIALOG YANG DIEVALUASI ===

{json.dumps(dialog_target, ensure_ascii=False, indent=2)}

Evaluasi 4 aspek dengan SCORE KETAT:

1. Konsistensi OCEAN (40 poin) — Cara bicara, emosi, pilihan kata harus mencerminkan OCEAN
2. Ketepatan emosi (30 poin) — Emosi harus sesuai dengan state "{konteks}" dan mood
3. Gaya bahasa usia (20 poin) — Bahasa harus sesuai kategori usia
4. Konsistensi karakter (10 poin) — NPC harus terasa konsisten

CONTOH SCORE:

- 90+: Dialog SANGAT HIDUP, personality jelas terasa, emosi kuat, berkesan
- 75-89: Dialog bagus, personality cukup terlihat, tapi masih bisa lebih baik
- 60-74: Dialog AMAN tapi generik, personality kurang jelas
- Di bawah 60: Dialog TIDAK sesuai personality

Refocus

Format JSON:

```
{
  "lulus": true/false,
  "score": <0-100>,
  "catatan": ["catatan 1", "catatan 2"],
  "saran_perbaikan": "saran spesifik (kosong jika lulus=true)"
}
```

Lulus jika `score >= {CRITIC_THRESHOLD}`.

Transition

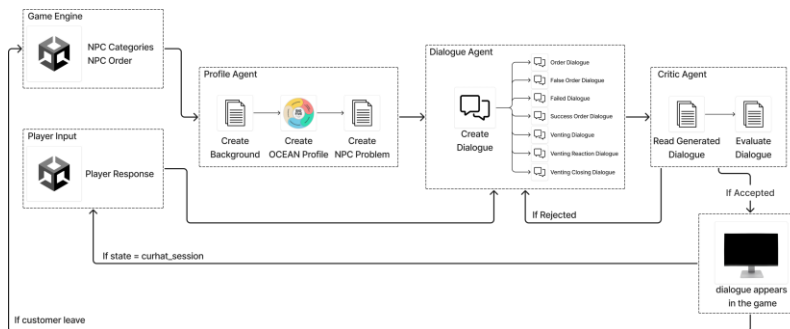
Kembalikan HANYA JSON valid

3.1.3.3 Analisis Alur Orkestrasi *Multi-Agent*

Dalam perancangan sistem *multi-agent* untuk game "*Tea'n Brew*", interaksi antar agent diatur untuk bekerja secara sekuensial guna memastikan kualitas dan konsistensi narasi yang dihasilkan. Alur kerja agen dimulai dari *Profile Agent*, dilanjutkan ke *Dialogue Agent*, dan diakhiri oleh *Critic Agent*.

Pendekatan *Pre-define*

Alur sistem yang dirancang ini menggunakan pendekatan *Pre-define*. Berdasarkan literatur, pendekatan *pre-define* adalah logika yang diekspresikan sebagai sekumpulan langkah atau cabang keputusan yang terbatas dan sudah diketahui urutannya sejak awal. Alasan penggunaan pendekatan *pre-define* pada sistem ini karena alur sistem yang sudah terdefinisi dari awal. Berikut ini adalah alur dari agent yang dapat dilihat pada **Gambar 3.8**.



Gambar 3.8 Alur sistem *multi agent*

Orchestration Framework

Framework yang dipilih dalam membangun sistem *multi agent* akan menggunakan LangGraph. *Framework* LangGraph memungkinkan alur *pre-define* tersebut direpresentasikan dalam bentuk graf berarah. Penggunaan LangGraph digunakan untuk mengoordinasikan agen-agen dengan tugas khusus [18]. Terdapat 2 komponen dalam *framework* langgraph tersebut yaitu:

1. Simpul (*Nodes*): Bertindak sebagai unit logika tempat *agent* (*Profile*, *Dialogue*, *Critic*) menjalankan tugasnya masing-masing.
2. Sisi (*Edges*): Mengatur transisi aliran data dari satu *agent* ke *agent* berikutnya sesuai urutan yang telah ditetapkan.

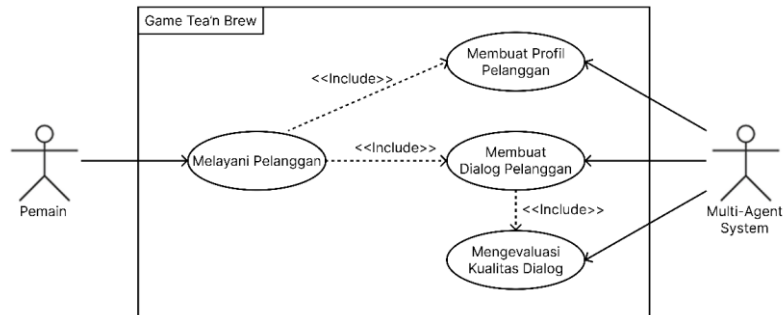
3.1.4 Analisis Kebutuhan Fungsional

Analisis kebutuhan fungsional merupakan proses analisis terhadap kebutuhan sistem dalam perancangan sistem *multi agent* yang akan dikembangkan dalam game “*Tea’n Brew*” agar dapat berjalan sesuai dengan kebutuhan pemain. Analisis kebutuhan fungsional yang dilakukan yaitu dengan membuat *use case diagram* dan *use case scenario* serta class diagram.

3.1.4.1 Use Case Diagram

Use case diagram ditujukan untuk mendeskripsikan interaksi pengguna dengan sistem. Hasil perancangan *use case diagram* dapat dilihat pada **Gambar 3.9**.

Commented [sr2]: tidak udah dibagi 2, cukup bilang use case diagram dan skenarionya



Gambar 3.9 Use Case Diagram

4. Definisi Aktor

Berdasarkan use case diagram pada **Gambar 3.9**, dideskripsikan aktor-aktor yang terlibat pada *use case diagram* tersebut. Berikut merupakan definisi dari masing-masing aktor yang dapat dilihat pada **Tabel 3.26**.

Tabel 3.26 Definisi Aktor

No	Aktor	Definisi
1	Pemain	Pemain yang memainkan game “Tea 'n Brew”
2	Multi Agent System	Sistem multi agent yang di host untuk menerima input dan menghasilkan output untuk game “Tea 'n Brew”

5. Definisi Use Case

Definisi use case berisi penjelasan dari setiap use case yang ada pada use case diagram pada **Gambar 3.9**. Berikut adalah definisi dari setiap use case yang dapat dilihat pada **Tabel 3.27**.

Tabel 3.27 Definisi Use Case

No	Use Case	Definisi
1	Melayani Customer	Pengguna memainkan game dan melayani <i>customer</i> dengan membuatkan pesanan dan mendengarkan curhatan <i>customer</i> .
2	Membuat Profil Customer	Sistem membuat <i>profile customer</i> setiap kali ada <i>customer</i> baru yang datang.
3	Membuat dialog Customer	Sistem membuat dialog berdasarkan jenis dan skenario game yang sedang berjalan.
4	Mengevaluasi Kualitas Dialog	Sistem menilai atau mengevaluasi dialog hasil dari dialogue agent untuk memastikan isi dialog sudah sesuai dengan <i>profile customer</i> .

3.1.4.2 Use Case Scenario

Use case Scenario merupakan penjelasan alur interaksi antara aktor dengan masing-masing use case. Berikut merupakan penjelasan dari masing-masing skenario use case.

1. Use Case Scenario Melayani Customer

Use case scenario ini menceritakan ketika pemain memainkan game “Tea’n Brew” dengan melayani *customer*, membuat minuman, dan mendengarkan curhatan *customer* yang dapat dilihat pada **Tabel 3.28**.

Tabel 3.28 Use Case Scenario Melayani Customer

Use Case Name	Melayani Customer	
Related Use case	Include: Profile Agent, Dialogue Agent, Critic Agent	
Goal In Context	Pemain melayani <i>customer</i> mulai dari menerima pesanan, membuat minuman, hingga sesi curhat untuk mendapatkan <i>score</i>	
Precondition	Pemain berada di Cafe Scene dan game masih berjalan (belum game over)	
Success End Condition	Pemain berhasil menyajikan minuman sesuai pesanan dan menyelesaikan sesi curhat, sehingga mendapat <i>score</i>	
Failed End Condition	ConditionPemain gagal membuat minuman sesuai pesanan hingga mencapai batas max_fail, <i>customer</i> pergi dan <i>score</i> berkurang -10	
Primary Actors	Pemain	
Secondary Actors	Multi Agent	
Trigger	Customer baru datang ke cafe (Customer Came)	
Main Flow	Step	Action
	1	Sistem memicu mekanik <i>randomize customer</i> dan meminta Multi Agent membuat profil serta dialog <i>customer</i>
	2	Sistem menampilkan <i>customer</i> beserta pesanan minumannya
	3	Pemain mengambil bahan sesuai resep di Diary Menu
	4	Pemain meracik minuman sesuai pesanan
	5	Pemain menghidangkan minuman ke <i>customer</i>
	6	Sistem memeriksa kesesuaian minuman dengan pesanan
	7	<i>Customer</i> menerima minuman lalu memulai sesi curhat
	8	Pemain memilih salah satu dari 3 pilihan respons terhadap curhatan
	9	Sistem menyesuaikan mood <i>customer</i> berdasarkan respons pemain
	10	Sistem memberi <i>score</i> akhir berdasarkan persentase mood akhir <i>customer</i>
Extensions	Step	Branching Action

	6	Jika minuman salah & belum max_fail → pesanan ditolak, pemain mengulang dari step 4
	6	Jika minuman salah & mencapai max_fail → customer pergi, score -10, use case berakhir gagal
	8-9	Diulang selama sesi curhat masih berlangsung

2. Use Case Scenario Membuat Profil Customer

Use case scenario ini menceritakan ketika multi agent system menjalankan *profile agent* untuk membuat *profile customer* yang dapat dilihat pada **Tabel 3.29**.

Tabel 3.29 Use Case Scenario Membuat Profile Customer

Use Case Name	Membuat Profil Customer	
Related Use case	Included by: Melayani Customer	
Goal In Context	Multi Agent menghasilkan profil customer baru secara dinamis (nama, latar belakang, masalah, score OCEAN)	
Precondition	Mekanik randomize customer terpicu	
Success End Condition	Profil customer berhasil dibuat dalam format JSON valid dan diteruskan ke Dialogue Agent	
Failed End Condition	Output tidak sesuai format JSON yang ditentukan	
Primary Actors	Multi Agent System	
Secondary Actors		
Trigger	Sistem mendeteksi perlunya customer baru (randomize customer)	
Main Flow	Step	Action
	1	Sistem mengirim system prompt & user prompt ke Profile Agent
	2	Profile Agent menentukan kategori usia & gender customer
	3	Profile Agent membuat nama dan latar belakang customer
	4	Profile Agent menentukan masalah/curhatan yang dibawa hari itu
	5	Profile Agent menghasilkan score kepribadian OCEAN (0–100) tiap dimensi
	6	Profile Agent mengembalikan hasil dalam format JSON
	7	Sistem meneruskan profil ke Dialogue Agent
Extensions	Step	Branching Action
	6	Jika output tidak sesuai format JSON → sistem meminta Profile Agent mengulang proses generate

3. Use Case Scenario Membuat dialog Customer

Use case scenario ini menceritakan ketika multi agent system menjalankan *dialogue agent* untuk membuat *dialog customer* yang dapat dilihat pada **Tabel 3.30**.

Tabel 3.30 Use Case Scenario Membuat Dialog Customer

Use Case Name	Membuat Dialog Customer
Related Use case	Included by: Melayani Customer; Input dari: Profile Agent

Goal In Context	Multi Agent (sebagai Actor) menghasilkan narasi dialog yang dinamis sesuai kepribadian OCEAN & latar belakang <i>customer</i>	
Precondition	Profil <i>customer</i> telah dibuat oleh <i>Profile Agent</i>	
Success End Condition	Dialog berhasil dibuat sesuai konteks dan diteruskan ke Critic Agent	
Failed End Condition	Dialog gagal dibuat sesuai format/konteks yang diminta	
Primary Actors	Multi Agent System	
Secondary Actors		
Trigger	Profil <i>customer</i> tersedia / sistem membutuhkan dialog untuk konteks tertentu (pesan, curhat, reaksi)	
Main Flow	Step	Action
	1	Sistem mengirim profil <i>customer</i> (nama, OCEAN, latar belakang, masalah) beserta konteks (state) ke Dialogue Agent
	2	Dialogue Agent menentukan gaya bahasa berdasarkan kategori usia
	3	Dialogue Agent menentukan emosi sesuai konteks/state
	4	Dialogue Agent menyusun narasi dialog sesuai <i>score</i> OCEAN
	5	Dialogue Agent mengembalikan dialog dalam format JSON
Extensions	6	Sistem meneruskan dialog ke Critic Agent untuk divalidasi
	Step	Branching Action
	5	Jika format JSON tidak sesuai → Dialogue Agent mengulang generate
	6	Jika Critic Agent menyatakan tidak lulus ($score < CRITIC_THRESHOLD$) → Dialogue Agent membuat ulang dialog berdasarkan saran_perbaikan

4. Use Case Scenario Mengevaluasi Kualitas Dialog

Use case scenario ini menceritakan ketika *multi agent system* menjalankan *Critic agent* untuk mengevaluasi dialog dari *dialogue agent* yang dapat dilihat pada **Tabel 3.31**.

Tabel 3.31 Use Case Scenario Membuat Dialog Customer

Use Case Name	Mengevaluasi Kualitas Dialog
Related Use case	Menilai output dari: Dialogue Agent
Goal In Context	Multi Agent mengevaluasi kualitas dialog sebagai <i>Quality Gate</i> sebelum ditampilkan ke pemain
Precondition	Dialogue Agent telah menghasilkan draf dialog
Success End Condition	Dialog dinyatakan lulus ($score \geq CRITIC_THRESHOLD$) dan diteruskan untuk ditampilkan
Failed End Condition	Dialog dinyatakan tidak lulus ($score < CRITIC_THRESHOLD$), perlu direvisi
Primary Actors	Multi Agent System

Secondary Actors		
Trigger	Dialogue Agent selesai membuat draf dialog	
Main Flow	Step	Action
	1	Sistem mengirim profil <i>customer</i> & draf dialog ke Critic Agent
	2	Critic Agent menilai konsistensi OCEAN (40 poin)
	3	Critic Agent menilai ketepatan emosi (30 poin)
	4	Critic Agent menilai gaya bahasa sesuai usia (20 poin)
	5	Critic Agent menilai konsistensi karakter (10 poin)
	6	Critic Agent menjumlah <i>score</i> total & menentukan status lulus/tidak
	7	Critic Agent mengembalikan hasil evaluasi (<i>score</i> , catatan, saran_perbaikan) dalam JSON
Extensions	8	Sistem meneruskan dialog yang lulus untuk ditampilkan ke pemain
	Step	Branching Action
	6	Jika <i>score</i> < CRITIC_THRESHOLD → sistem mengirim dialog kembali ke Dialogue Agent untuk direvisi (lihat Extension Dialogue Agent)

3.1.4.3 Analisis Class

Analisis class dilakukan untuk mengidentifikasi class yang dibutuhkan dalam menerapkan sistem multi-agent pada game *Tea 'n Brew*. Identifikasi class mengacu pada kebutuhan fungsional yang telah dijelaskan melalui use case dan skenario use case. Berdasarkan hasil analisis, diperlukan beberapa class baru pada sisi Unity untuk menghubungkan alur permainan dengan sistem multi-agent serta untuk menyimpan data output multi-agent. Beberapa class yang dibutuhkan yaitu:

1. MultiAgentFlowController

Class MultiAgentFlowController dibutuhkan untuk mengatur alur interaksi customer yang melibatkan sistem multi-agent. Class ini menghubungkan alur pelanggan, data pelanggan, data resep, dan hasil yang diperoleh dari sistem multi-agent. Berikut ini adalah variable yang dibutuhkan pada class MultiAgentFlowController yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class MultiAgentFlowController

Variabel	Tipe Data	Keterangan
customerAdapter	MultiAgentCustomerAdapter	Menyimpan penghubung ke fungsi sistem multi-agent.
customerProfile	CustomerProfile	Menyimpan data pelanggan yang sedang digunakan.
requestedRecipe	TeaRecipe	Menyimpan resep minuman yang sedang dipesan.

customerName	string	Menyimpan nama pelanggan aktif.
failCount	integer	Menyimpan jumlah kesalahan penyajian.
currentMood	integer	Menyimpan kondisi mood pelanggan.
isCurhatActive	boolean	Menyimpan status sesi curhat.

2. MultiAgentCustomerAdapter

Class MultiAgentCustomerAdapter dibutuhkan sebagai perantara antara pengendali alur permainan dan bagian komunikasi sistem multi-agent. Class ini mengelola data sesi serta hasil profile dan dialog yang digunakan selama interaksi pelanggan. Berikut ini adalah variable yang dibutuhkan pada class MultiAgentCustomerAdapter yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class MultiAgentCustomerAdapter

Variabel	Tipe Data	Keterangan
multiAgentBridge	MultiAgentBridge	Menyimpan penghubung ke bagian komunikasi sistem.
sessionId	string	Menyimpan identitas sesi pelanggan.
profileData	ProfileData	Menyimpan profil dinamis pelanggan.
dialogData	DialogData	Menyimpan dialog dinamis yang diterima.
evaluationData	EvaluationData	Menyimpan hasil evaluasi jawaban pemain.
isGenerating	boolean	Menyimpan status proses pembuatan data.
currentRound	integer	Menyimpan ronde curhat yang sedang berjalan.

3. MultiAgentBridge

Class MultiAgentBridge digunakan untuk menghubungkan Unity dengan sistem multi-agent. Class ini menampung konfigurasi komunikasi dan kondisi proses pertukaran data. Berikut ini adalah variable yang dibutuhkan pada class MultiAgentBridge yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class MultiAgentCustomerAdapter

Variabel	Tipe Data	Keterangan
serverUrl	string	Menyimpan alamat server sistem multi-agent.
timeout	integer	Menyimpan batas waktu proses komunikasi.
isConnected	boolean	Menyimpan status hubungan dengan server.
lastError	string	Menyimpan informasi kesalahan komunikasi terakhir.

4. ProfileResponse

Class ProfileResponse merupakan kelas respons yang menyimpan hasil dari profile agent. Class ini akan menampung data profile pelanggan yang nanti akan digunakan untuk agent lainnya. Berikut ini variabel yang dibutuhkan pada class ProfileResponse yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class ProfileResponse

Variabel	Tipe Data	Keterangan
session_id	string	Identitas sesi pelanggan.
nama	string	Nama pelanggan yang dihasilkan.
background	string	Latar belakang atau informasi personal pelanggan.
masalah_hari_ini	string	Masalah yang sedang dialami pelanggan pada hari tersebut.
ocean	OceanData	Data kepribadian pelanggan berdasarkan lima dimensi OCEAN.
max_fails	int	Jumlah maksimum kesalahan pesanan yang dapat ditoleransi.

5. OceanData

Class OceanData menyimpan nilai kepribadian pelanggan berdasarkan model Big Five atau OCEAN. Data kepribadian ini digunakan oleh agent untuk menyesuaikan karakter, gaya bicara, dan respons pelanggan. Berikut ini variabel yang diperlukan pada class OceanData yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class OceanData

Variabel	Tipe Data	Keterangan
openness	int	Tingkat keterbukaan pelanggan terhadap pengalaman dan ide baru.
conscientiousness	int	Tingkat keteraturan, kedisiplinan, dan tanggung jawab pelanggan.
extraversion	int	Tingkat keterbukaan sosial dan energi interaksi pelanggan.
agreeableness	int	Tingkat keramahan, empati, dan kecenderungan bekerja sama.
neuroticism	int	Tingkat kepekaan pelanggan terhadap tekanan atau emosi negatif.

6. DialogueResponse

Class DialogueResponse menyimpan dialog yang diterima game dari dialogue agent. Class ini digunakan untuk menyimpan berbagai kondisi interaksi pelanggan, termasuk pemesanan, pesanan salah, pelanggan marah, pesanan berhasil, dan sesi curhat. Berikut ini variabel yang dibutuhkan pada class DialogueResponse yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class DialogueResponse

Variabel	Tipe Data	Keterangan
session_id	string	Identitas sesi pelanggan yang menghasilkan dialog.
dialog_state	string	Keadaan atau konteks dialog.
dialog_text	string	Isi dialog yang ditampilkan kepada pemain.
minuman_dipesan	string	Nama minuman yang dipesan pelanggan.
pilihan_jawaban	PilihanJawaban	Daftar pilihan jawaban yang dapat dipilih pemain.
critic_skor	int	Skor evaluasi dialog dari Critic Agent.
critic_lulus	bool	Menunjukkan apakah dialog dinyatakan memenuhi kriteria.

Variabel	Tipe Data	Keterangan
total_ronde	int	Jumlah total ronde dalam sesi curhat.
ronde_sekarang	int	Nomor ronde curhat yang sedang berlangsung.

7. PilihanJawaban

Class PilihanJawaban menyimpan pilihan respons yang ditampilkan kepada pemain pada sesi curhat. Class ini tidak menyimpan jawaban akhir pemain secara permanen, melainkan menyimpan opsi yang ditawarkan oleh dialogue agent. Berikut ini variabel yang dibutuhkan pada class PilihanJawaban yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class PilihanJawaban

Variabel	Tipe Data	Keterangan
id	int	Nomor identitas pilihan jawaban.
teks	string	Isi jawaban yang ditampilkan kepada pemain.
nada	string	Kategori nada dari pilihan jawaban.

8. EvaluateData

Class EvaluateData menyimpan hasil evaluasi jawaban pemain yang dikirim oleh critic agent. Berikut ini variabel yang dibutuhkan pada class EvaluateData yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Variabel Class EvaluateResponse

Variabel	Tipe Data	Keterangan
session_id	string	Identitas sesi pelanggan.
nada	string	Nada jawaban pemain berdasarkan hasil evaluasi.
mood_sebelum	int	Nilai mood pelanggan sebelum jawaban diproses.
mood_sesudah	int	Nilai mood pelanggan setelah jawaban diproses.
reaksi_npc	string	Reaksi NPC terhadap jawaban pemain.
riwayat_entry	RiwayatEntry	Catatan interaksi pada ronde tersebut.
total_ronde	int	Jumlah keseluruhan ronde curhat.
ronde_sekarang	int	Nomor ronde yang baru saja diproses.

9. RiwayatEntry

Class RiwayatEntry menyimpan catatan interaksi antara NPC dan pemain dalam satu ronde curhat. Kelas ini merupakan kelas yang secara langsung menyimpan jawaban pemain sebagai bagian dari riwayat percakapan. Berikut ini variabel yang dibutuhkan pada class RiwayatEntry yang dapat dilihat pada **Tabel 3.x**.

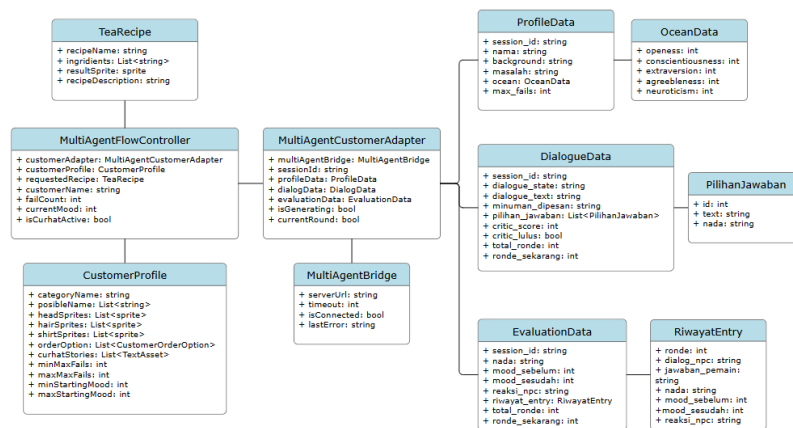
Tabel 3.x Variabel Class RiwayatEntry

Variabel	Tipe Data	Keterangan
ronde	int	Nomor ronde tempat interaksi terjadi.

Variabel	Tipe Data	Keterangan
dialog_npc	string	Dialog atau pernyataan NPC pada ronde tersebut.
jawaban_pemain	string	Jawaban aktual yang diberikan pemain.
nada	string	Nada jawaban pemain.
mood_sebelum	int	Mood NPC sebelum menerima jawaban pemain.
mood_sesudah	int	Mood NPC setelah menerima jawaban pemain.

3.1.4.4 Class Diagram

Class diagram dibuat untuk menampilkan struktur sistem game yang menunjukkan hubungan antar class. Berikut ini hasil class diagram yang dapat dilihat pada Gambar 3.x.



Commented [sr3]: class masuknya ke perancangan method juga ditulis, jangan setengah2

Gambar 3.x Class Diagram Game Tea'n Brew

3.1.5 Analisis Kebutuhan Non Fungsional

Analisis kebutuhan non fungsional merupakan proses analisis terhadap kebutuhan di luar sistem untuk mengoperasikan game “Tea’n Brew”. Analisis kebutuhan non fungsional dibagi menjadi 3 macam yaitu, analisis kebutuhan perangkat pikir, analisis kebutuhan perangkat keras, dan analisis kebutuhan perangkat lunak. Berikut merupakan penjelasan dari masing-masing analisis kebutuhan non fungsional.

3.1.5.1 Analisis Kebutuhan Perangkat Pikir

Analisis kebutuhan perangkat pikir merupakan proses analisis terhadap karakteristik pengguna yang dibutuhkan untuk memainkan game “Tea’n Brew”.

Pada bagian ini akan diuraikan kebutuhan-kebutuhan yang berhubungan dengan spesifikasi pengguna. Berikut merupakan uraian analisis kebutuhan perangkat pikir:

- 1 Memiliki pemahaman dalam memainkan *game visual novel*.
- 2 Memiliki pengalaman menggunakan internet
- 3 Memiliki kemampuan untuk membaca
- 4 Tingkat pendidikan sekolah menengah hingga mahasiswa.

3.1.5.2 Analisis Kebutuhan Perangkat Keras

Pada bagian analisis kebutuhan perangkat keras dilakukan analisis terhadap spesifikasi perangkat keras yang akan digunakan pada pengembangan sistem *multi agent* untuk *game "Tea'n Brew"*. Berikut merupakan perangkat keras yang akan digunakan:

1. *Platform*: Windows 10 or newer
2. *CPU*: Intel i3 gen 8 or better
3. *Memory*: 8GB
4. *Storage*: 512MB
5. Internet

3.1.5.3 Analisis Kebutuhan Perangkat Lunak

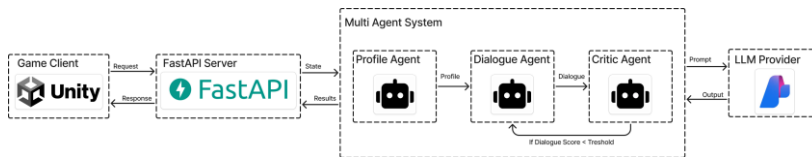
Analisis kebutuhan perangkat lunak merupakan proses analisis terhadap spesifikasi perangkat lunak yang akan dikembangkan. Kebutuhan perangkat lunak yang akan dikembangkan adalah sistem operasi Windows.

3.2 Perancangan Sistem

Perancangan sistem menjelaskan bagaimana komponen permainan Tea'n Brew diintegrasikan dengan sistem multi-agent untuk menghasilkan profile dan dialog pelanggan secara dinamis. Sistem dirancang menggunakan arsitektur client-server. Unity berperan sebagai klien, sedangkan aplikasi berbasis FastAPI berperan sebagai server yang menghubungkan permainan dengan sistem multi-agent dan penyedia Large Language Model atau LLM. Tahapan perancangan sistem akan dibagi menjadi 2 tahapan yaitu perancangan arsitektur sistem dan perancangan server.

3.2.1 Perancangan Arsitektur Sistem

Berikut ini adalah rancangan arsitektur lengkap sistem keseluruhan yang dapat dilihat pada **Gambar 3.x**.



Gambar 3.x Arsitektur Sistem Game Tea'n Brew

3.2.2 Perancangan Server

FastAPI merupakan pusat komunikasi antara Unity dan sistem multi-agent. Server menyediakan sejumlah endpoint yang merepresentasikan operasi yang dapat diminta oleh permainan. Berikut adalah tabel endpoint yang dapat dilihat pada **Tabel 3.x**.

Tabel 3.x Daftar endpoint sistem

No.	Metode	Endpoint	Fungsi
1	GET	/health	Memeriksa status server.
2	POST	/generate_profile	Membuat profil dan sesi pelanggan.
3	POST	/generate_pesanan	Menghasilkan dialog pemesanan.
4	POST	/generate_salah	Menghasilkan reaksi pesanan salah.
5	POST	/generate_marah	Menghasilkan dialog pelanggan marah.
6	POST	/generate_berhasil	Menghasilkan dialog ketika pesanan berhasil.
7	POST	/start_curhat	Memulai sesi curhat.
8	POST	/next_curhat_round	Melanjutkan ronde curhat.
9	POST	/evaluate_answer	Mengevaluasi jawaban pemain dan mengubah mood.
10	GET	/session/{session_id}/summary	Mengambil ringkasan sesi.
11	DELETE	/session/{session_id}	Menghapus sesi.

1. Endpoint Health Check
2. Endpoint Pembuatan Profil
3. Endpoint Dialog
4. Endpoint Memulai Sesi Curhat
5. Endpoint Ronde Curhat Berikutnya
6. Endpoint Ringkasan
7. Endpoint Penghapusan Session

Commented [sr4]: perbaiki lagi arsitektur

Commented [sr5]: harusnya tidak selevel dengan arsitektur

DAFTAR PUSTAKA

- [1] K. Cameron, “Narrative and gameplay design in the story-driven videogame: A case study on The Last of Us,” M.A. thesis, Auckland Univ. of Technology, 2017.
- [2] J. M. Domingues, V. Filipe, A. Carita, and V. Carvalho, “Understanding the Impact of Perceived Challenge on Narrative Immersion in Video Games: The Role-Playing Game Genre as a Case Study,” *Information*, vol. 15, no. 6, p. 294, 2024.
- [3] X. Wang et al., “ClueCart: Supporting Game Story Interpretation and Narrative Inference from Fragmented Clues,” in *Proc. CHI '25*, 2025.
- [4] N. Vickery, N. Tancred, P. Wyeth, and D. Johnson, “Directing narrative in gameplay: Player interaction in shaping narrative in The Witcher 3,” in *Proc. OzCHI '18*, 2018.
- [5] F. Garavaglia, R. A. Nobre, L. A. Ripamonti, D. Maggiorini, and D. Gadia, “Moody5: Personality-biased agents to enhance interactive storytelling in video games,” in *2022 IEEE Conference on Games (CoG)*, 2022, pp. 175–182.
- [6] S. Buongiorno, L. J. Klinkert, T. Chawla, Z. Zhuang, and C. Clark, “PANGeA: Procedural Artificial Narrative using Generative AI for Turn-Based Video Games,” *arXiv preprint arXiv:2404.19721*, 2024.
- [7] G. S. V. R. Maraffi, “Towards a Serious Game Experience Model Validation,” in *International Conference on Games and Learning Alliance*, 2023.
- [8] J. L. Sherry, “Measuring narrative engagement,” *Media Psychology*, vol. 6, no. 3, pp. 323-344, 2004.
- [9] J. S. Park et al., “Generative Agents: Interactive Simulacra of Human Behavior,” in *Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023.

- [10] A. Jovan, A. Harahap, and R. Indrasvari, "Android-Based Endless Runner Game Learning Media Innovations to Improve Students' Mathematical Literacy," *Jurnal Eduscience*, vol. 13, no. 1, pp. 128-135, Feb. 2026.
- [11] K. Oygardslia, "The Educational Potential of Visual Novel Games: Principles for Design," *Journal of the Ritsumeikan Center for Game Studies*, vol. 2, pp. 59-70, Mar. 2020.
- [12] H. Warpefelt, "The Non-Player Character: Exploring the Believability of NPC Presentation and Behavior," Ph.D. dissertation, Dept. Comput. Syst. Sci., Stockholm Univ., Stockholm, Sweden, 2016.
- [13] Berryman, J., & Ziegler, A. (2025). *Prompt Engineering for LLMs: The Art and Science of Building Large Language Model-Based Applications* (1st ed.). O'Reilly Media, Inc.
- [14] M. Albada, *Building Applications with AI Agents: Designing and Implementing Multiagent Systems*. Santa Rosa, CA: O'Reilly Media, Inc., 2025.
- [15] J. G. Bond, *Introduction to Game Design, Prototyping, and Development: From Concept to Playable Game with Unity and C#*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2022.
- [16] E. Matthes, *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*, 3rd ed. San Francisco, CA, USA: No Starch Press, 2023.
- [17] Artificial Analysis. "AI models & providers language model benchmark analysis." artificialanalysis.ai. <https://artificialanalysis.ai/>
- [18] A. M. Althaf and J. Talburt, "Multi-Agent RAG Framework for Entity Resolution: Advancing Beyond Single-LLM Approaches with Specialized Agent Coordination," *Computers*, vol. 14, no. 12, Art. no. 525, 2025, doi: 10.3390/computers14120525.
- [19] S. Mehri and M. Eskenazi, "Unsupervised evaluation of interactive dialog with DialoGPT," in *Proc. 21st Annu. Meeting Special Interest Group Discourse Dialogue (SIGDIAL)*, 1st Virtual Meeting, Jul. 2020, pp. 225–235, doi: 10.18653/v1/2020.sigdial-1.28.

- [20] J. Li, M. Galley, C. Brockett, G. Spithourakis, J. Gao, and B. Dolan, “A persona-based neural conversation model,” in Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (ACL), vol. 1: Long Papers, Berlin, Germany, Aug. 2016, pp. 994–1003, doi: 10.18653/v1/P16-1094.
- [21] S. Zhang, E. Dinan, J. Urbanek, A. Szlam, D. Kiela, and J. Weston, “Personalizing dialogue agents: I have a dog, do you have pets too?,” in Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (ACL), vol. 1: Long Papers, Melbourne, Australia, Jul. 2018, pp. 2204–2213, doi: 10.18653/v1/P18-1205.
- [22] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in Proc. 2023 Conf. Empirical Methods Natural Language Processing (EMNLP), Singapore, Dec. 2023, pp. 2511–2522, doi: 10.18653/v1/2023.emnlp-main.153.
- [23] Z. Xia, J. Xu, Y. Zhang, and H. Liu, “A survey of uncertainty estimation methods on large language models,” arXiv:2503.00172, 2025, doi: 10.48550/arXiv.2503.00172.
- [24] J. Gu et al., “A survey on LLM-as-a-Judge,” arXiv preprint arXiv:2411.15594, Nov. 2024.

LAMPIRAN A

Review dan Rating Pemain

Username	Rate	Review
RyujiKazu	4	Udh bagus banget tapi aku ngerasa dialognya ngulang terus jadinya gamenya kayak kurang hidup berasa monoton
ZulfiFazhar	3	Game nya menarik, tapi saya merasakan konten nya berulang disitu saja. Sebaiknya dibuat lebih variatif agar player tidak mudah bosan
Glacier Tazer	2	Banyak minusnya nih game, artnya gaada yang special, musiknya b aja, mekaniknya itu loh terlalu mirip coffee talk. saranku kasih progression atau minimal percakapannya nyambung lah, ga ngulang-ngulang atau monoton begitu.
Handaaa	4	Kalo bisa tambahin variasi percakapannya, soalnya tadi pas main makin lama kok ngulang2. Overall udah bagus sih paling update minor kalo game ini lebih fokus ke percakapan karakter kecuali mau improve gameplaynya juga (๖´˘˘ˆ๖)
Kris Bolt	1	Jelek gamenya, gaada ceritanya, NPCnya cuma ngulang ngulang cerita yang sama doang.
Wira Kusuma	4	Keren gamenya, musiknya juga cocok banget. kalo variasi <i>customernya</i> dibanyakin bakal lebih asik sih
yeriko winwin	3	mirip coffee talk gamenya. ini emang NPCnya ga inget apa-apa ya dari curhat sebelumnya? kurang sih di bagian itu
ninja copilot	3	gamenya ok, tambahin story mode mungkin soalnya yg sekarang gaada ceritanya
just TINE	2	bosenin, dialognya ga nyambung dan kesannya ngulang ngulang aja
Gabriel Lino	3	keren gamenya, minus dialognya kaku aja
kevin yopin	4	Gamenya bagus, cuma setelah mainin lama dialog yang sama muncul lagi. Coba di perbanyak variasinya mungkin bakal mendingan
6162201035-dev	4	Pengulangan dialog membuat interaksi antar karakter terasa membosankan dan monoton.
KursusHub	2	-
Rapapapaps	3	-
Salt Studio	4	-
andrianbaros	3	dialognya terlalu berulang
hilmy270301	3	-
justine62	3	game ini terlalu repetitif sehingga tidak ada "sense of progress" dari game ini

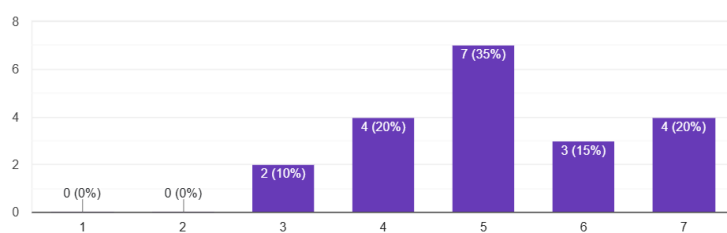
LAMPIRAN B

Kuisisioner GEQ dan NES

Saya merasa benar-benar terbenam dalam dunia permainan selama sesi ini

[Salin diagram](#)

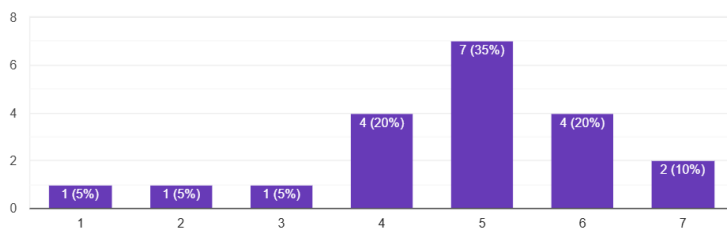
20 jawaban



Saya sering lupa waktu ketika sedang bermain

[Salin diagram](#)

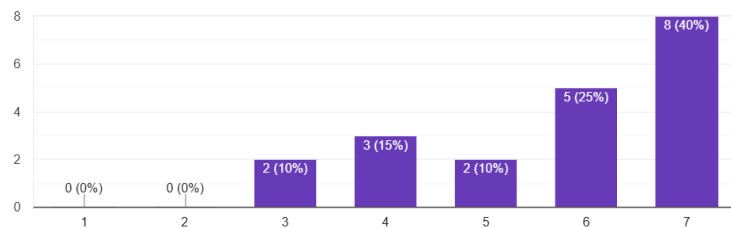
20 jawaban



Visual dan suasana permainan membuat saya merasa 'hadir' dalam cerita

[Salin diagram](#)

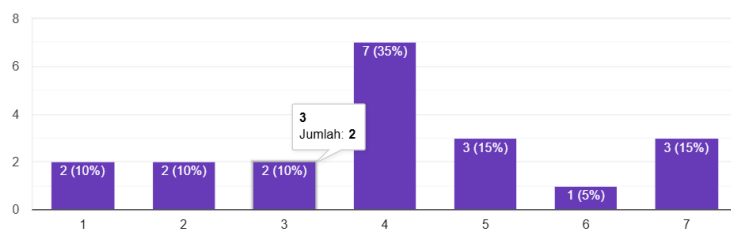
20 jawaban



Saya mudah terganggu oleh hal di luar permainan

[Salin diagram](#)

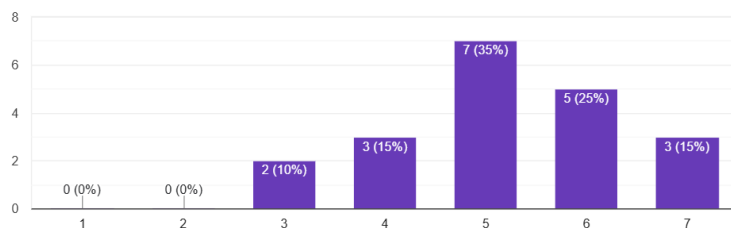
20 jawaban



Saya merasa suasana permainan menyatu dengan cerita sehingga sulit melepaskan perhatian

[Salin diagram](#)

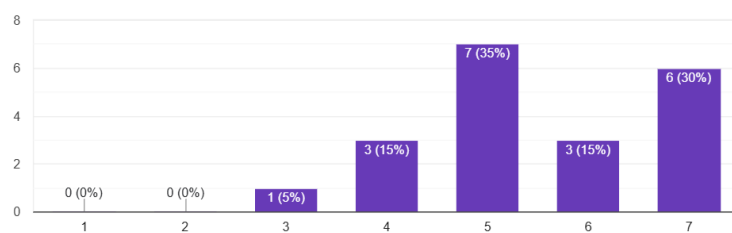
20 jawaban



Saya merasa sangat terfokus pada tugas yang harus saya lakukan di permainan

 Salin diagram

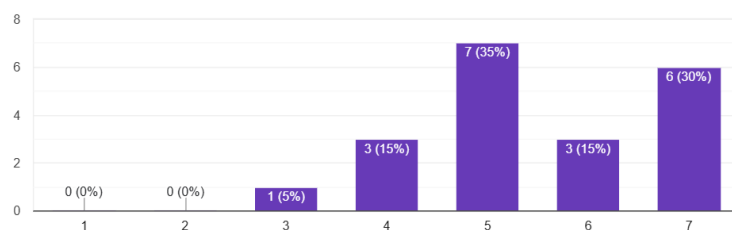
20 jawaban



Saya merasa sangat terfokus pada tugas yang harus saya lakukan di permainan

 Salin diagram

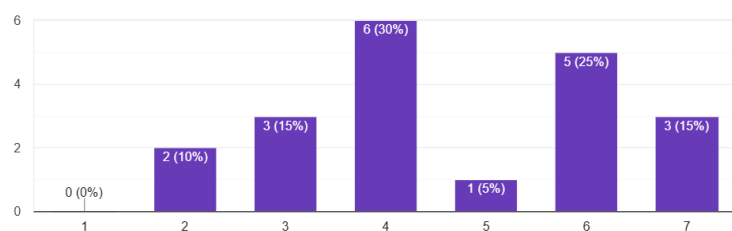
20 jawaban



Saya kehilangan rasa waktu karena terlibat dalam aktivitas permainan

 Salin diagram

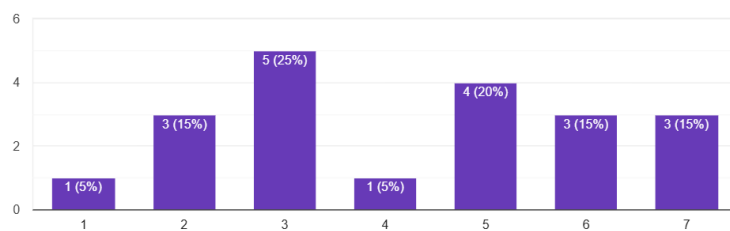
20 jawaban



Saya mudah merasa bosan selama sesi ini

 Salin diagram

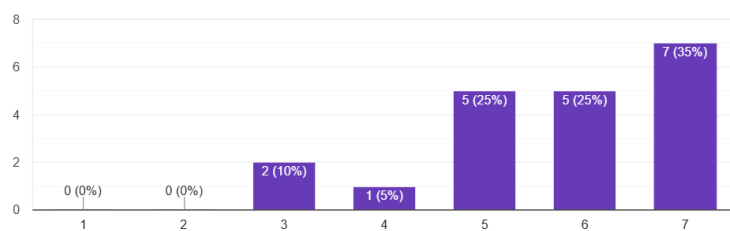
20 jawaban



Aktivitas permainan terasa lancar dan alami saat saya bermain.

 Salin diagram

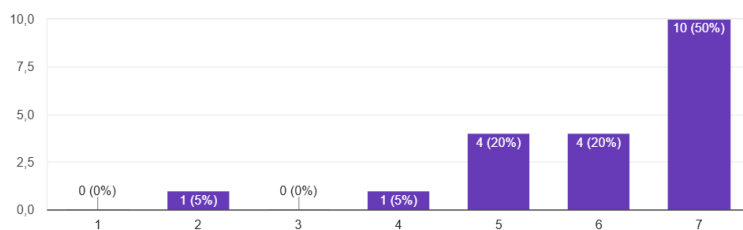
20 jawaban



Saya merasa mampu mengerjakan mekanik dasar dengan baik

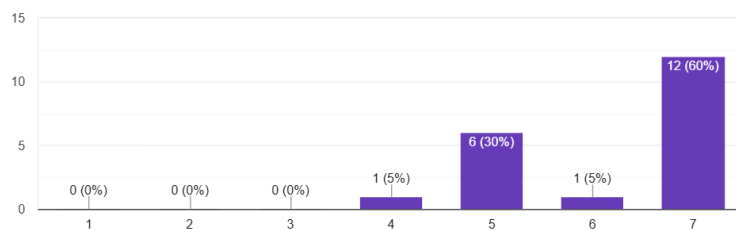
 Salin diagram

20 jawaban



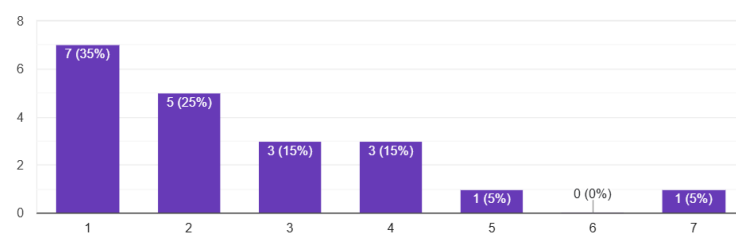
Saya yakin dengan kemampuan saya dalam menghadapi tantangan permainan [Salin diagram](#)

20 jawaban



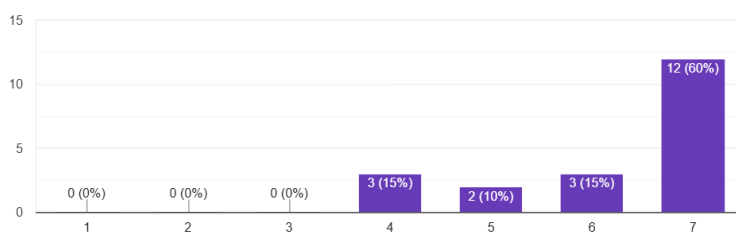
Saya merasa frustrasi karena tidak cukup terampil [Salin diagram](#)

20 jawaban



Saya sering berhasil menyelesaikan tugas yang diberikan permainan [Salin diagram](#)

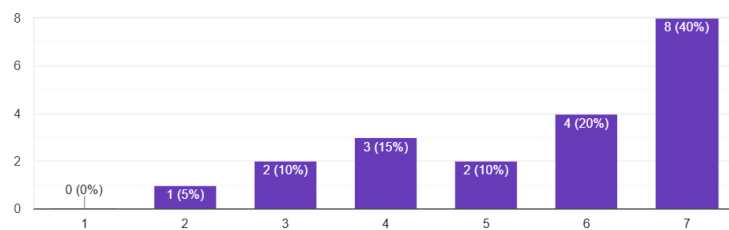
20 jawaban



Saya merasa progres/keberhasilan saya jelas terlihat selama bermain

[Salin diagram](#)

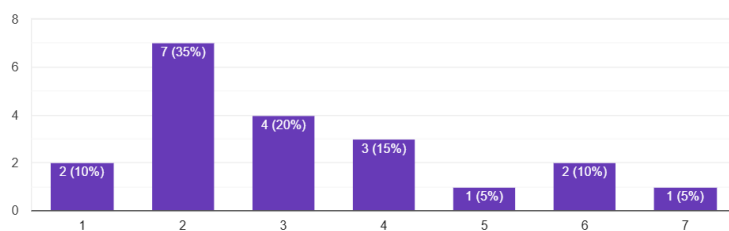
20 jawaban



Saya merasa kesal/frustrasi terhadap aspek permainan

[Salin diagram](#)

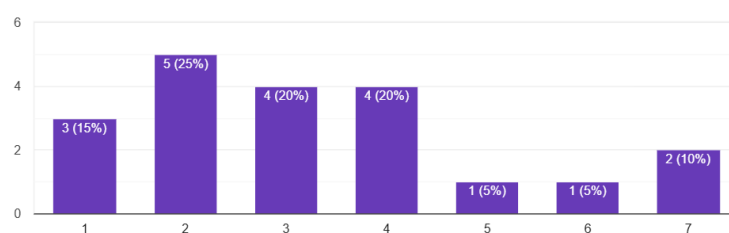
20 jawaban



Permainan membuat saya merasa stres pada beberapa momen

[Salin diagram](#)

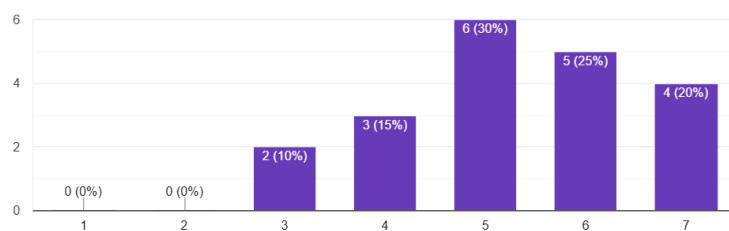
20 jawaban



Saya merasa tenang saat bermain

[Salin diagram](#)

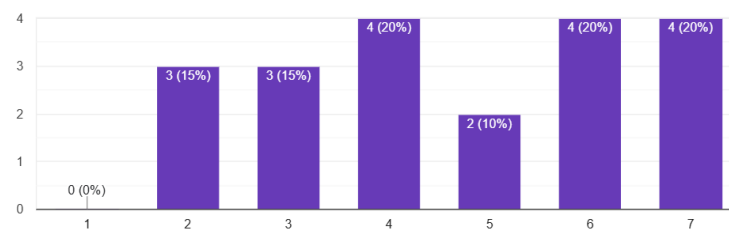
20 jawaban



Saya merasa ada bagian permainan yang mengganggu kenikmatan saya

[Salin diagram](#)

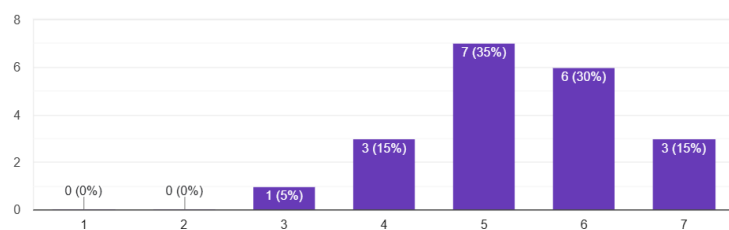
20 jawaban



Saya merasa puas dengan pengalaman bermain setelah sesi ini

[Salin diagram](#)

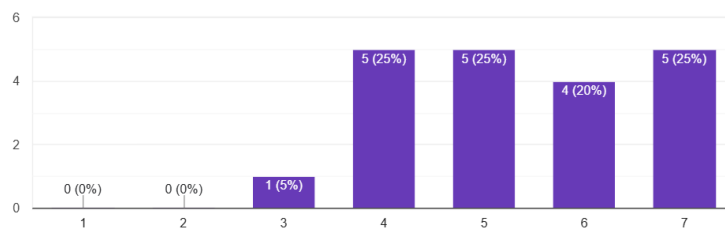
20 jawaban



Bermain membuat saya merasa senang/terhibur

[Salin diagram](#)

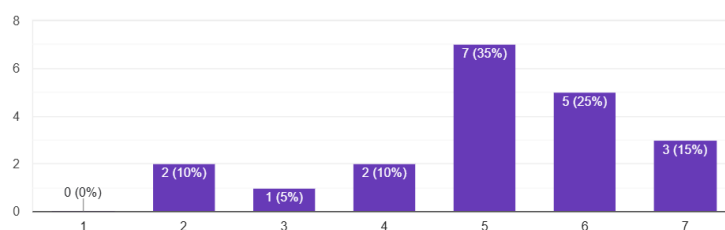
20 jawaban



Saya merasa terinspirasi/tertarik oleh cerita dan karakter

[Salin diagram](#)

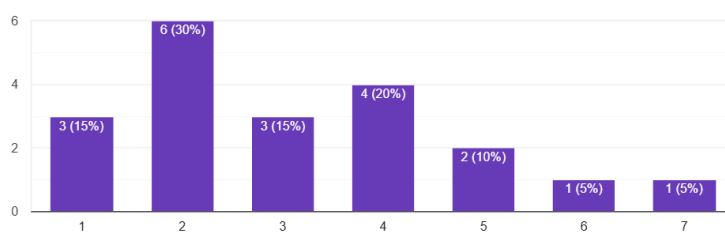
20 jawaban



Saya merasa kecewa atau tidak puas setelah bermain

[Salin diagram](#)

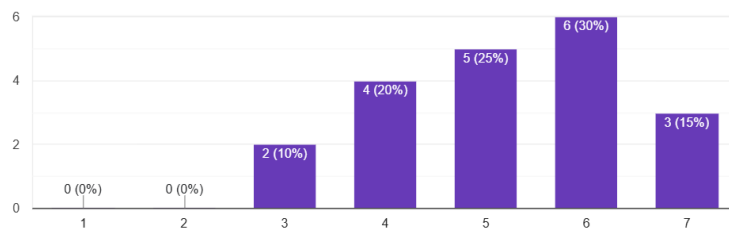
20 jawaban



Saya cenderung mengingat pengalaman positif dari sesi ini

 Salin diagram

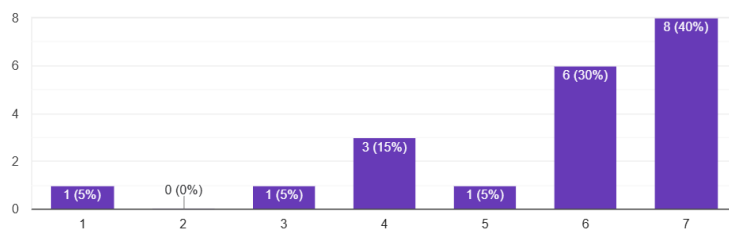
20 jawaban



Alur cerita dan motivasi karakter mudah saya pahami

 Salin diagram

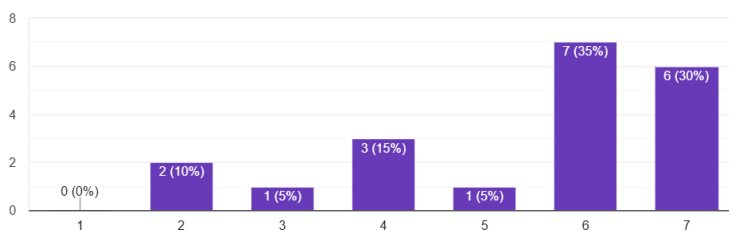
20 jawaban



Saya dapat mengikuti sebab-akibat kejadian dalam cerita dengan jelas

 Salin diagram

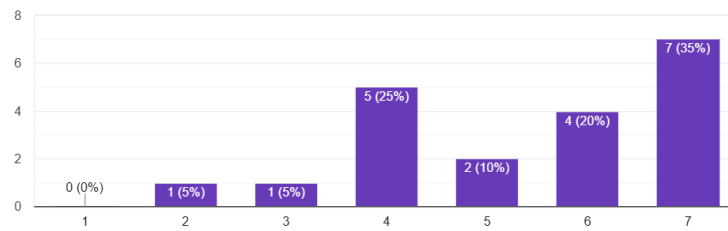
20 jawaban



Saya dapat membayangkan urutan peristiwa dalam cerita dengan baik

[Salin diagram](#)

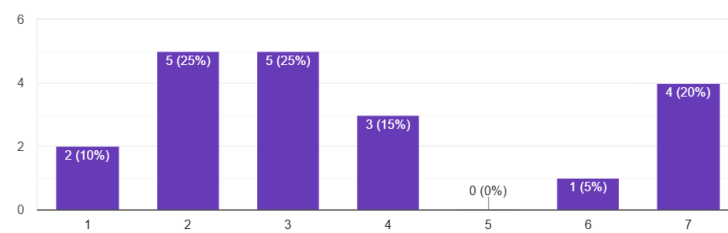
20 jawaban



Beberapa bagian cerita terasa membingungkan bagi saya

[Salin diagram](#)

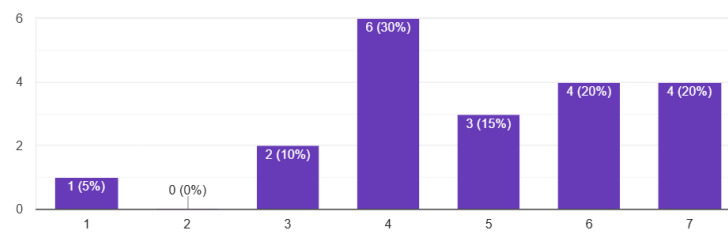
20 jawaban



Saya merasa cerita tersusun logis dan koheren

[Salin diagram](#)

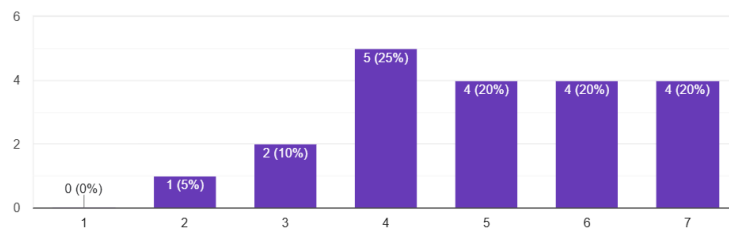
20 jawaban



Perhatian saya sepenuhnya tertuju pada cerita selama bermain

[Salin diagram](#)

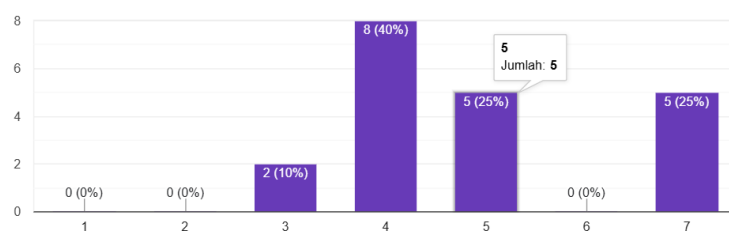
20 jawaban



Saya jarang teralihkan oleh hal lain ketika cerita berlangsung

[Salin diagram](#)

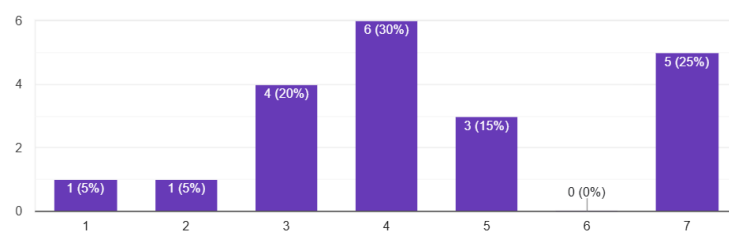
20 jawaban



Saya merasa sangat terlibat secara mental dalam percakapan dengan NPC

[Salin diagram](#)

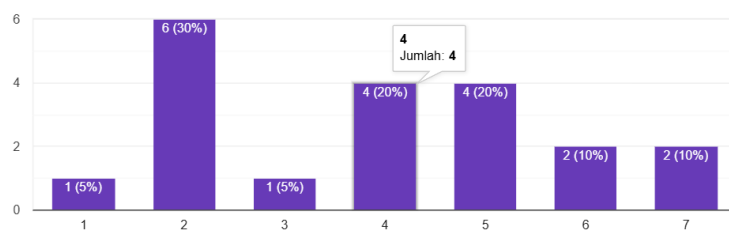
20 jawaban



Pikiran saya sering melayang ke hal lain saat dialog berlangsung

[Salin diagram](#)

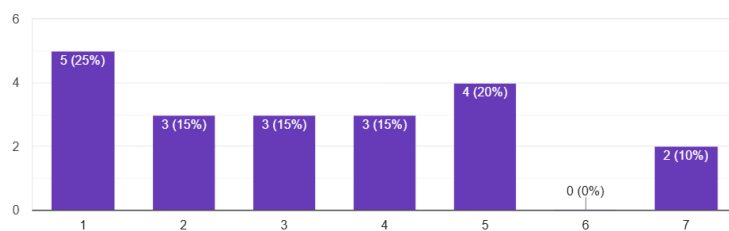
20 jawaban



Saya merasa sulit berhenti memikirkan apa yang baru saja terjadi di cerita

[Salin diagram](#)

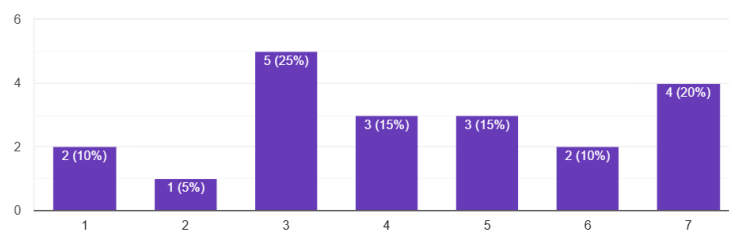
20 jawaban



Saya merasa terikat secara emosional dengan satu atau lebih pelanggan/NPC

[Salin diagram](#)

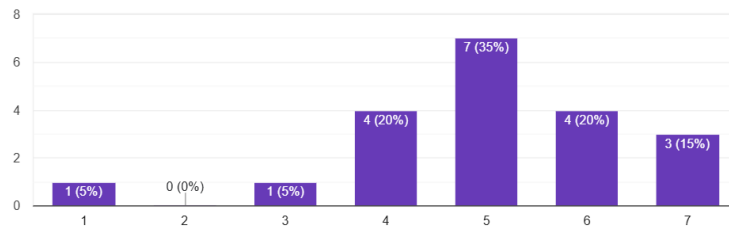
20 jawaban



Saya merasakan empati terhadap masalah yang dibagikan NPC

[Salin diagram](#)

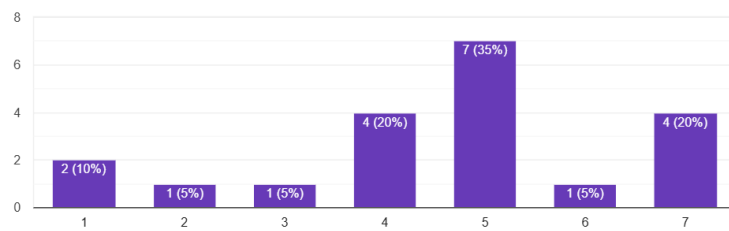
20 jawaban



Saya merasa sedih atau senang sesuai dengan nasib karakter dalam cerita

[Salin diagram](#)

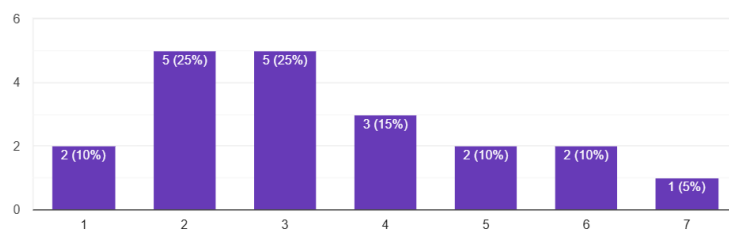
20 jawaban



Saya merasa acuh tak acuh terhadap apa yang diceritakan NPC

[Salin diagram](#)

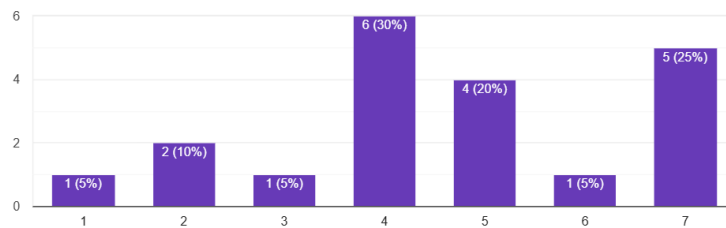
20 jawaban



Saya terbawa perasaan oleh dialog dan reaksi NPC

[Salin diagram](#)

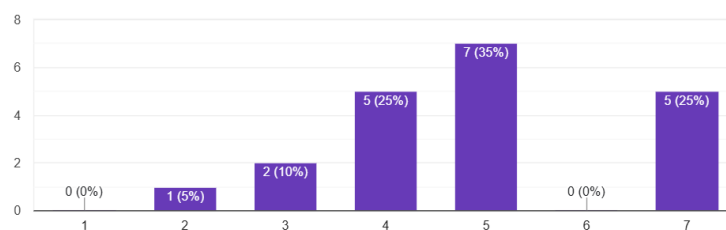
20 jawaban



Saat cerita berlangsung, saya merasa seolah berada di lingkungan permainan

[Salin diagram](#)

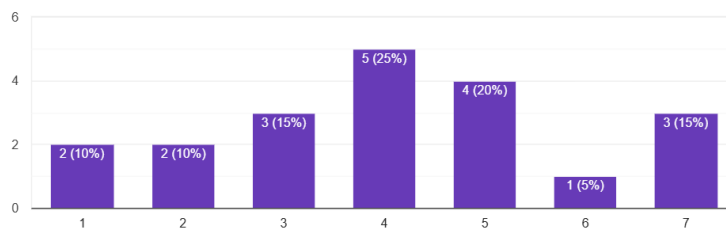
20 jawaban



Saya merasa 'terbawa' ke dalam dunia cerita sehingga lingkungan nyata terlupakan

[Salin diagram](#)

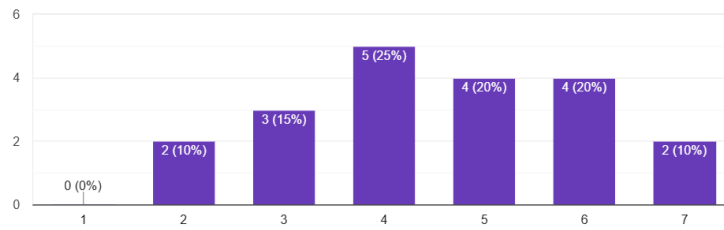
20 jawaban



Pengalaman cerita terasa nyata sehingga saya ikut merasakan suasana NPC

[Salin diagram](#)

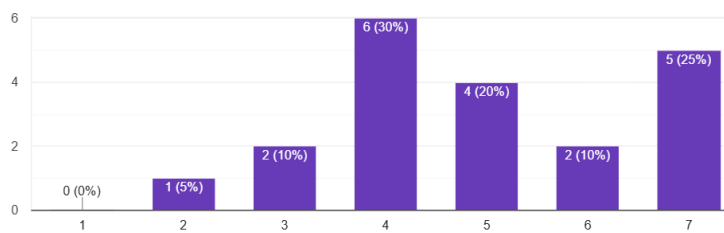
20 jawaban



Saya sering sadar sepenuhnya akan lingkungan di sekitar saya selagi bermain

[Salin diagram](#)

20 jawaban



Saya merasa ikut mengalami kejadian yang terjadi dalam cerita

[Salin diagram](#)

20 jawaban

